LAPORAN TUGAS AKHIR APLIKASI MOBILE

Contoh Kesalahan Umum
Activity, Sequence, Usecase, Class Diagram

Disusun Untuk Memenuhi Tugas Akhir Mata Kuliah
Aplikasi Mobile



Disusun Oleh Kelompok 8 :

Abdul Rozak Ainurrofiq (221344033)
Febryan Adi Putra (221344043)

3B – TEKNIK TELEKOMUNIKASI DAN NIRKABEL

Tanggal Penyusunan Laporan : 12/06/2025
Tanggal Pengumpulan Laporan : 17/06/2025

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI BANDUNG

2024-2025

DAFTAR ISI

DAFTAR ISI 1
1. TUJUAN TUGAS AKHIR 2
2. TEORI PENDAHULUAN 2
2.1. Activity Diagram 2
2.2. Sequence Diagram 4
2.3. Usecase Diagram 6
2.4. Class Diagram 8
3. KESALAHAN UMUM DALAM PEMBUATAN ACTIVIY DIAGRAM 10
4. KESALAHAN UMUM DALAM PEMBUATAN SEQUENCE DIAGRAM 21
5. KESALAHAN UMUM DALAM PEMBUATAN USECASE DIAGRAM 34
6. KESALAHAN UMUM DALAM PEMBUATAN CLASS DIAGRAM 49
7. KESIMPULAN 62

1. TUJUAN TUGAS AKHIR

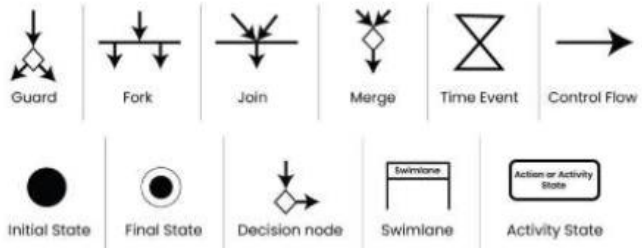
1) Mengetahui klasifikasi dan jenis kesalahan umum activity diagram, sequence diagram, use case diagram, dan class diagram
2) Memahami dan memberi contoh kesalahan umum pada activity diagram, sequence diagram, use case diagram, dan class diagram

2. TEORI PENDAHULUAN

2.1. Activity Diagram

Activity Diagram merupakan bagian dari Unified Modeling Language (UML) yang digunakan untuk menggambarkan alur proses aktivitas dalam sistem secara visual. Menurut Booch, Rumbaugh, dan Jacobson (2005), Activity Diagram berfungsi untuk memodelkan perilaku dinamis sistem, khususnya dalam menampilkan urutan aktivitas, keputusan, dan kondisi paralel dalam sebuah proses (Booch et al., 2005). Activity Diagram sangat berguna untuk:

- Menggambarkan suatu proses bisnis atau alur kerja sistem secara visual.
- Mendetilkan alur kerja sistem dalam fungsi sistem secara keseluruhan yang menjadi tahapan selanjutnya setelah pembuatan use case.
- Media untuk menggambarkan cara kerja sistem kepada system designer, system analyst dan stackholder yang memahami kerja sistem secara umum.
- Menyederhanakan system workflow dengan mudah dalam bentuk visual yang disepakati.



Komponen Dasar Activity Diagram

Menurut Arlow dan Neustadt (2005), dalam menyusun Activity Diagram, terdapat beberapa komponen penting (Arlow & Neustadt, 2005), yaitu:

- **Merge Node**

  Menggabungkan beberapa jalur masuk menjadi satu alur tunggal.

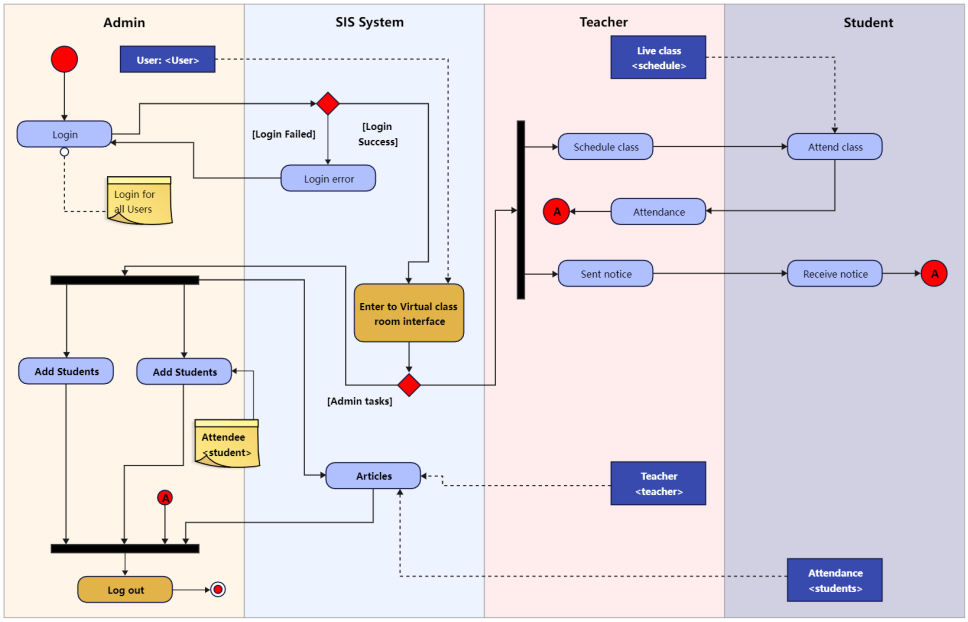
- **Fork Node dan Join Node**

  Digunakan untuk menggambarkan aktivitas paralel, dengan garis tebal horizontal atau vertikal.

- **Final Node**

  Menyatakan akhir dari alur aktivitas, digambarkan dengan lingkaran hitam ganda.

Swimlane (jalur renang) Activity Diagram adalah bentuk pengembangan dari activity diagram yang mengelompokkan aktivitas ke dalam kolom atau baris (swimlane) untuk menunjukkan aktor atau sistem mana yang bertanggung jawab atas aktivitas tersebut ([Dennis, Wixom, dan Tegarden, 2015]). Swimlane digunakan untuk memperjelas alur kerja antar berbagai pihak dalam satu proses bisnis atau sistem, sehingga pembagian tanggung jawab dan interaksi antar aktor dapat terlihat lebih jelas ([Larman, 2004]).



Contoh Implementasi Swimlane

2.2. Sequence Diagram

Sequence Diagram merupakan Merupakan diagram yang digunakan untuk menggambarkan langkah-langkah penyelesaian suatu proses yang dilaksanakan oleh aktor atau objek terhadap sistem dalam batasan waktu secara vertikal maupun horisontal. Koneksi horisontal merupakan hubungan atau relasi antar objek atau aktor terhadap sistem atau sebaliknya. Koneksi vertikal merupakan penjelasan aktivasi proses yang dilaksanakan oleh suatu objek atau aktor dalam rentang waktu tertentu.

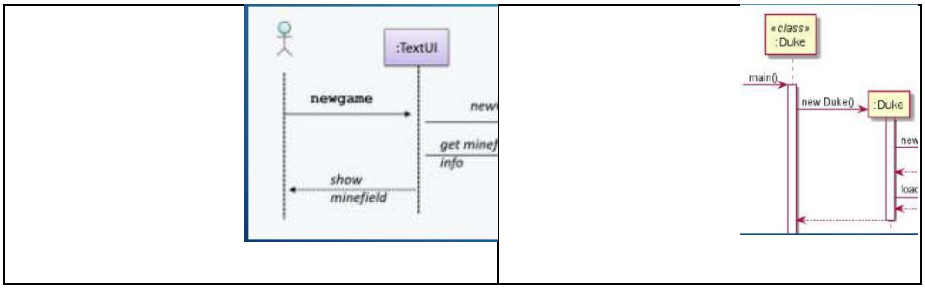
Berikut merupakan komponen komponen dari sequence diagram

- **Objek dan Aktor**

  Objek ditentukan setidaknya dari 1 usecase yang boleh jadi akan terpakai di usecase lainnya Penentuan objek akan sangat mudah ditentukan dari langkah-langkah yang menjadi keterangan di usecase description (narrative). Aktor atau Aktor eksternal merupakan entitas diluar sistem yang melakukan aktivasi terhadap usecase tersebut, hal ini dapat berupa pengguna atau sistem eksternal di luar sistem yang dibuat.

- **Lifeline dan Activation Box**

  Lifeline Merupakan garis vertikal putus-putus yang menjadi penentu waktu aktivasi dari suatu objek atau kelas atau fungsi pada sistem yang berinteraksi dengan fungsi lain atau entitas lainnya selama skenarionya dilaksanakan dari kiri ke kanan. Bentuk lifeline dan Activation Box dapat digambarkan berupa persegi panjang tipis yang ditandai dengan garis putus-putus.



Contoh Lifeline dan Activation Box

- **Pesan (Messages)**

  Berbetuk panah yang bertugas untuk melaksanakan interaksi antar suatu objek atau entitas atau fungsi pada sistem

- **Condition, Loop, Alternative and Optional (Alt / Opt)**

  Merupakan runtunan proses pada interaksi yang dilaksanakan antar entitas, objek dan atau komponen lainnya berupa fungsi atau subsistem

- **Return Value**

  Merupakan reaksi dari suatu pesan atau interaksi yang dilakukan oleh objek, entitas atau fungsi lainnya sebagai penanda suatu balasan dari aksi sebelumnya.

- **Sequence Flow**

  Merupakan urutan aliran pesan hasil dari interaksi antar objek, aktor, entitas atau fungsi lain sesuai dengan penentuan waktu aktivasinya.

- **Self-Message**

  Merupakan pesan yang disampaikan kepada dirinya sendiri (objek, entitas, fungsi atau subsistem) sebagai penanda bahwa terdapat aksi lainnya yang perlu dilaksanakan namun tidak berasal dari entitas, objek atau fungsi atau subsistem lain.

Sequence diagram memiliki beberapa keunggulan dalam pengembangan sistem, seperti memperjelas proses berdasarkan use case diagram, memudahkan pembuatan sistem secara modular, serta mempermudah pemahaman bagi programmer dan system designer. Diagram ini juga membantu dalam evaluasi sistem, karena menggambarkan interaksi antar komponen secara rinci. Selain itu, sequence diagram menjadi bagian dari dokumentasi perancangan, yang dapat digunakan sebagai panduan atau referensi dalam pengembangan lanjutan.

2.3. Usecase Diagram

Usecase Diagram merupakan diagram yang menggambarkan fungsionalitas yang diharapkan dari sebuah sistem. Yang ditekankan adalah “apa” yang diperbuat sistem, dan bukan “bagaimana”. Sebuah use case merepresentasikan sebuah interaksi antara aktor dengan sistem.

1) Use case



Use case dibuat berdasar keperluan actor, merupakan “apa” yang dikerjakan system, bukan “bagaimana” system mengerjakannya. Use case diberi nama yang menyatakan apa hal yang dicapai dari hasil interaksinya dengan actor. Use casedinotasikan dengan gambar (horizontal ellipse). Use case biasanya menggunakan kata kerja. Nama use case boleh terdiri dari beberapa kata dan tidak boleh ada 2 use case yang memiliki nama yang sama.

2) Actor



Actor menggambarkan orang, system atau external entitas / stakeholder yang menyediakan atau menerima informasi dari system. Actor menggambarkan sebuah tugas/peran dan bukannya posisi sebuah jabatan. Actor memberi input atau menerima informasi dari system. Actor biasanya menggunakan Kata benda. Tidak boleh ada komunikasi langsung antar actor. Indikasi <<system>> digunakan untuk sebuah actor yang merupakan sebuah system. Adanya actor bernama “Time” yang mengindikasikan scheduled events (suatu kejadian yang terjadi secara periodik/bulanan). Actor utama diletakkan pada pojok kiri atas dari diagram

3) Association

Associations bukan menggambarkan aliran data/informasi. Associations digunakan untuk menggambarkan bagaimana actor terlibat dalam use case. Ada 4 jenis relasi yang bisa timbul pada use case diagram.

- Association antara actor dan use case

  Ujung panah pada association antaraactor dan use case mengindikasikan siapa/apa yang meminta interaksi dan bukannya mengindikasikan aliran data. Sebaiknya gunakan Garis tanpa panah untuk association antara actor dan use case. Association antara actor dan use case yang menggunakan panah terbuka untuk mengindikasikan bila actor berinteraksi secara pasifdengan system anda

- Association antara use case

  <<include>> termasuk didalam use case lain (required) / (diharuskan). Use case yang terkait dengan <<include>> menandakan keharusan atau kewajiban bahwa jika suatu use case digunakan use case yang terhubung harus digunakan juga.

  <<extend>> perluasan dari use case lain jika kondisi atau syarat terpenuhi. Jika usecase terkait dengan <<extend>> maka penggunaan use case yang terhubung itu bersifat opsional atau condisional dan tidak wajib.

- Generalization/Inheritance antara use case

  Generalization/inheritance digambarkan dengan sebuah garis berpanah tertutup pada salah satu ujungnya yang menunjukkan lebih umum. Gambaran generalization/inheritance antara use case secara vertical dengan inheriting use case dibawah base/parent use case. Generalization/inheritance dipakai ketika ada sebuah keadaan yang lain sendiri/perlakuan khusus (single condition)

- Generalization/Inheritance antara actors

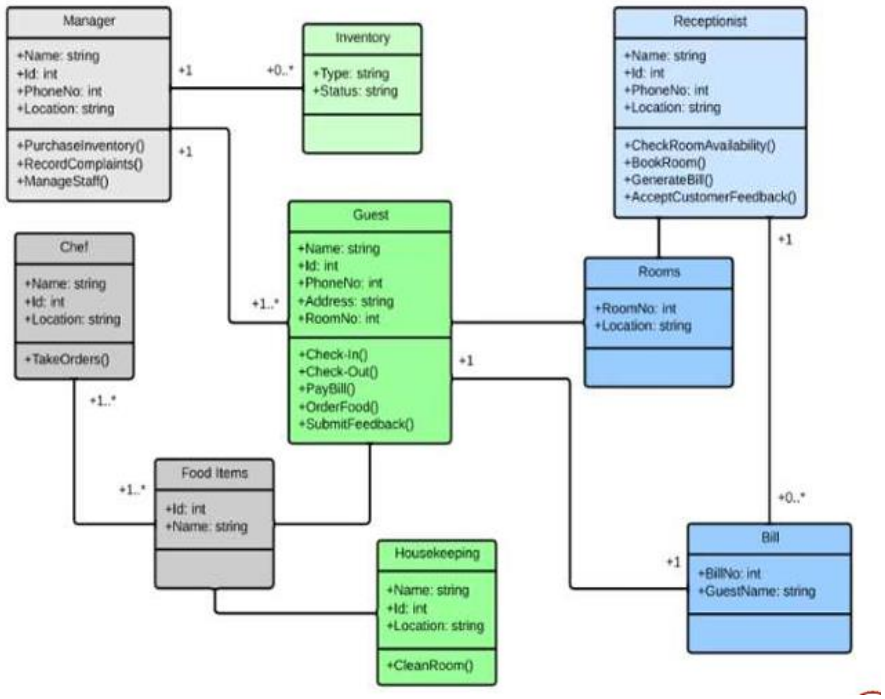
  Gambaran generalization/inheritance antara actors secara vertical dengan inheriting actor dibawah base/parent use case.

2.4. Class Diagram

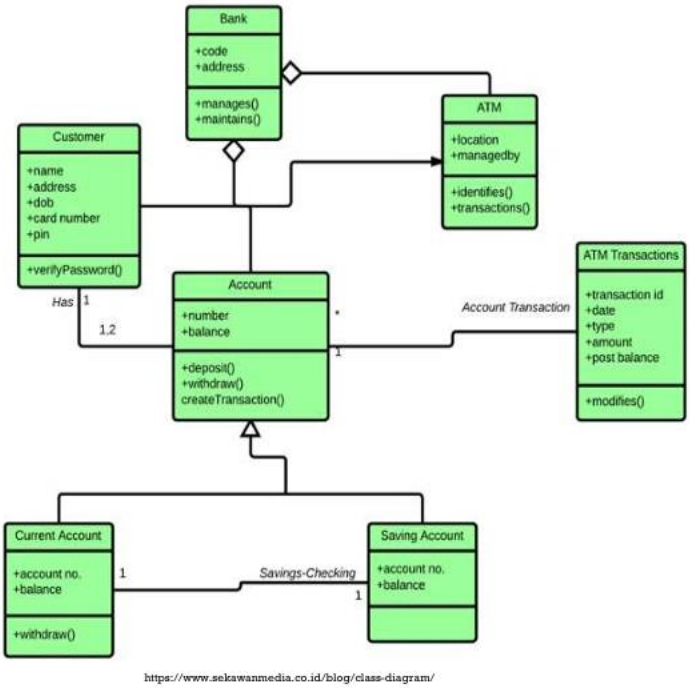
Menurut Martin Fowler (2004), Class Diagram adalah diagram struktur yang memperlihatkan kelas–kelasdalamsuatusistem beserta atribut dan operasi (metode) yang dimilikinya. Fowler menekankan bahwa class diagram juga menggambarkan relasi statis—seperti asosiasi, agregasi, komposisi,dan generalisasi—yang mengikat objek-objek dalam sistem menjadi sebuah arsitektur logis.

Class Diagram adalah diagram yang menggambarkanstrukturstatisdarisistem yang dirancang, termasuk kelas-kelas, atribut-atribut, dan hubungan antara kelas-kelas. (Grady Booch et al.,2007).

Dalam konteks rekayasa perangkat lunak, class diagram berperan sebagai cetakan biru (blueprint) yang menunjukkan bagaimana objek-objek dalam sistem akan dikelola dan dihubungkan satu sama lain. Ini sangat berguna dalam tahap analisisdandesainsistemkarena mampumenjembatanikomunikasi antara analis, perancang, dan pengembang (Ian Sommerville, 2011).



Implementasi Class diagram pada sistem operasional hotel

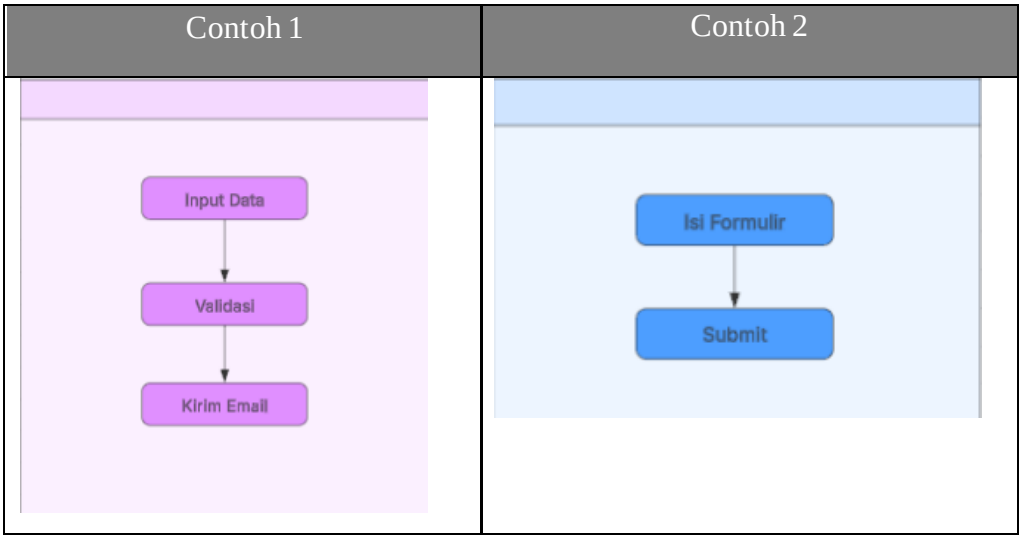


Implementasi Class diagram pada sistem mesin ATM

3. KESALAHAN UMUM DALAM PEMBUATAN ACTIVIY DIAGRAM

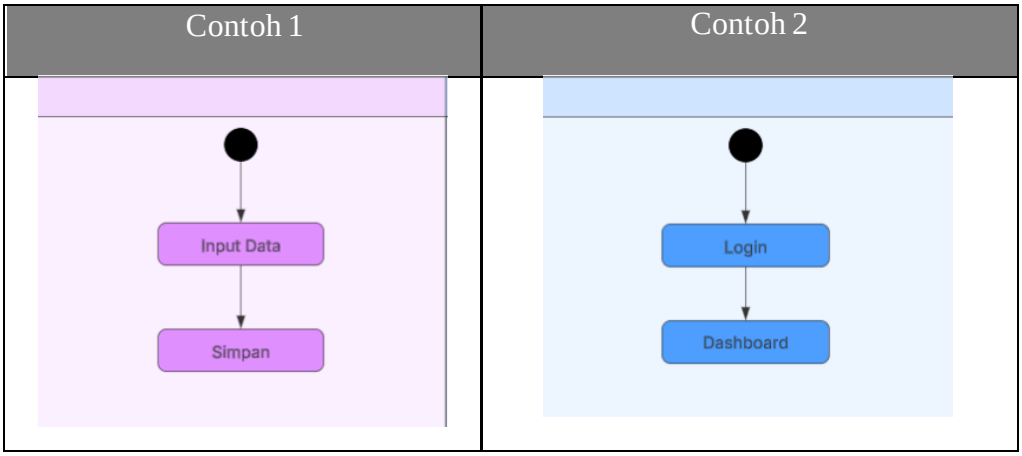
3.1.Kesalahan Struktural Dasar Diagram

a. Tidak menggunakan initial node



Initial node (●) adalah penanda titik awal aktivitas. Tanpa ini, pembaca tidak tahu dari mana proses dimulai. Seharusnya ada Initial node di awal input data dan isi formulir

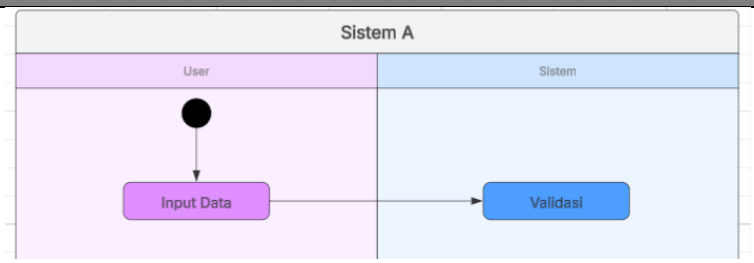
b. Mengabaikan Penggunaan Final Node



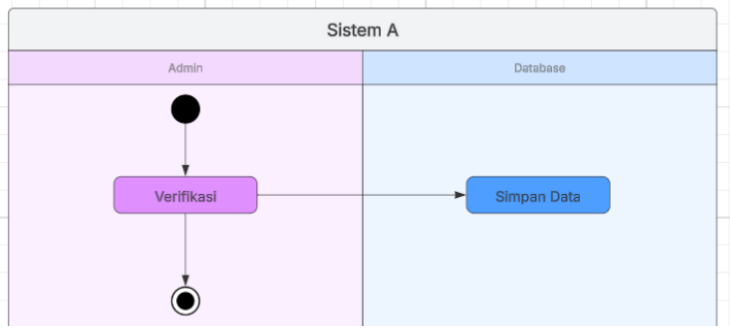
Final node (◎) menunjukkan proses berakhir. Tanpa final node, diagram tampak menggantung dan tidak jelas apakah aktivitas sudah selesai. Ini bisa menyebabkan asumsi yang salah saat implementasi sistem. Harusnya ditutup dengan final node setelah simpan dan dashboard yang menandakan aktivitas selesai

c. Mengabaikan Aktivitas Akhir Dalam Swimlane Berbeda

Contoh 1



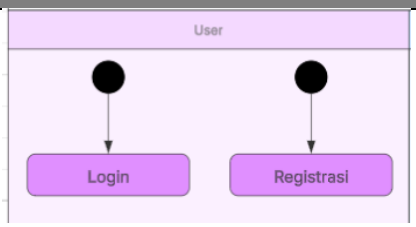
Contoh 2



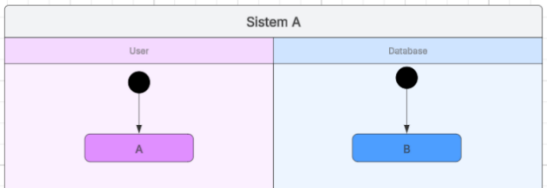
Jika aktivitas terakhir dilakukan oleh aktor lain (misalnya sistem, bukan user), maka final node harus berada di swimlane aktor tersebut. Meletakkan final node di tempat yang salah membuat pemetaan peran jadi tidak akurat.

d. Menggunakan Lebih Dari Satu Initial Node

Contoh 1

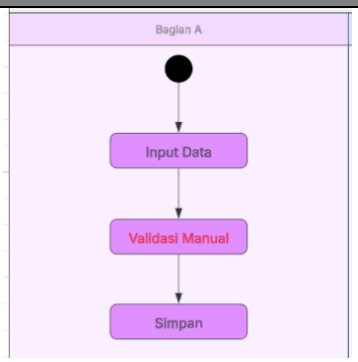
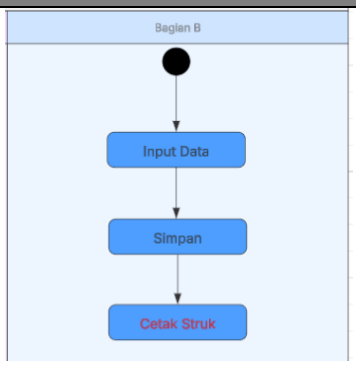


Contoh 2



Activity diagram hanya boleh punya satu titik awal. Jika ada dua initial node, diagram jadi membingungkan—apakah kedua alur berjalan bersamaan atau hanya salah satu yang dipakai? Gunakan fork node atau keputusan awal jika ingin memecah jalur.

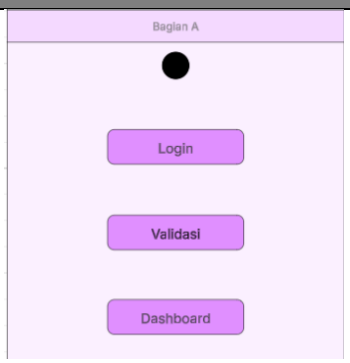
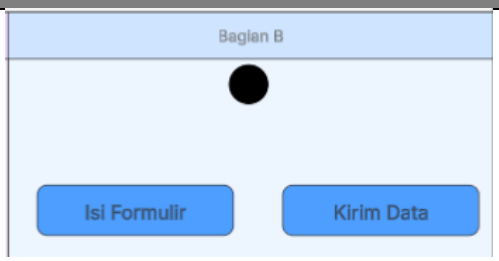
e. Tidak Konsisten Antara Use Case dan Activity Diagram

Contoh 1	Contoh 2
	

Activity diagram seharusnya merupakan detail dari skenario pada use case. Jika aktivitas yang ada tidak disebut dalam use case, maka konsistensinya diragukan dan bisa menyebabkan kesalahan dalam analisis sistem. Pada contoh 1 seharusnya setelah input data dilanjut dengan simpan tetapi pada activity diagram ada penambahan validasi manual yang seharusnya tidak boleh dilakukan, untuk contoh 2 seharusnya berhenti di simpan

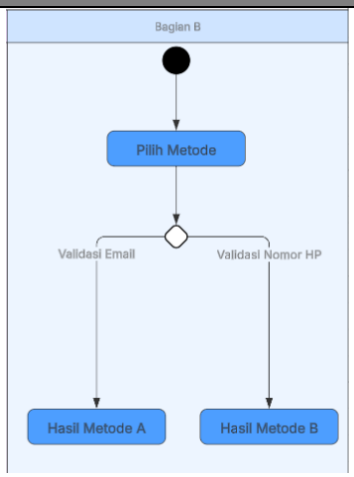
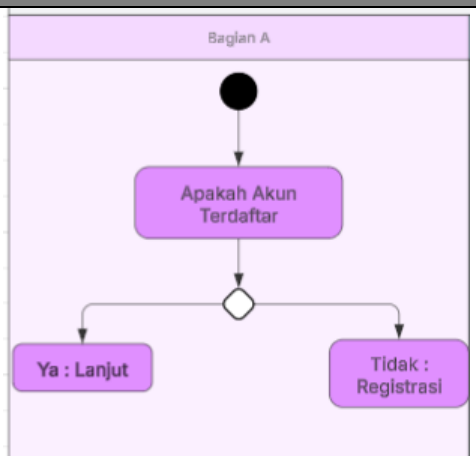
3.2.Kesalahan Dalam Alur Kontrol

a. Menghubungkan Action Langsung Tanpa Control Flow

Contoh 1	Contoh 2
	

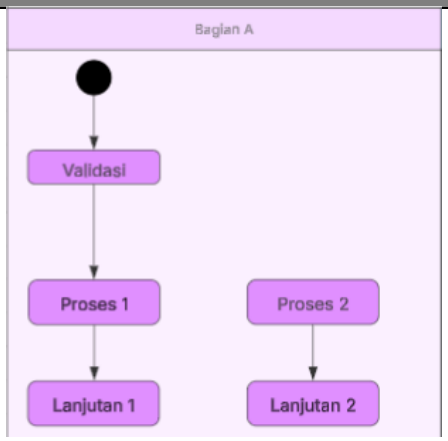
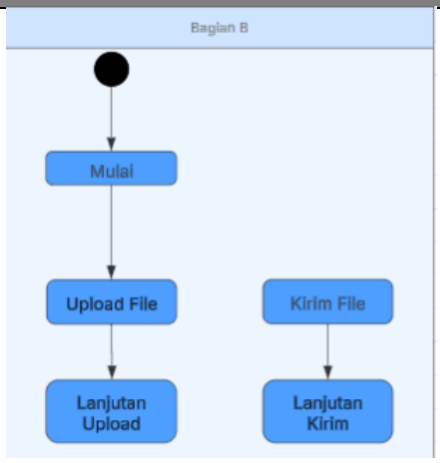
Tanpa control flow (panah arah), urutan aktivitas menjadi ambigu. Diagram seolah-olah hanya daftar kegiatan, bukan aliran proses. Ini mengaburkan logika eksekusi sistem.

b. Menggunakan Terlalu Banyak Keputusan Tanpa Merge Node

Contoh 1	Contoh 2
	

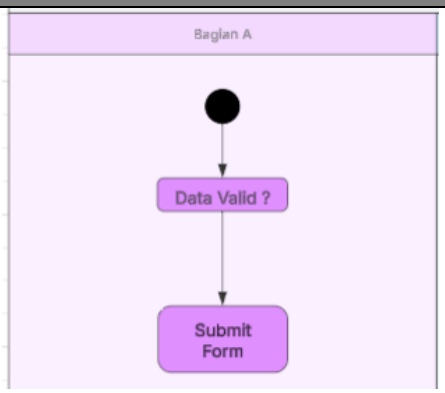
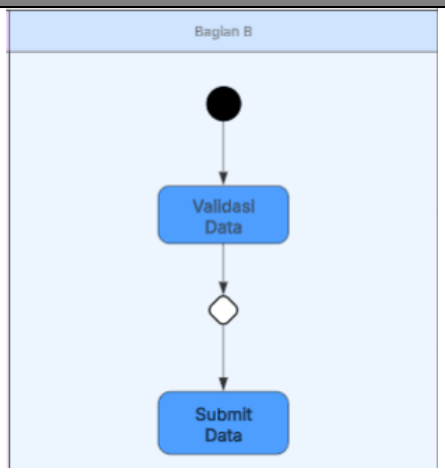
Tanpa merge node, jalur bercabang tidak dikembalikan ke alur utama. Ini membuat diagram terpecah dan susah dilacak, serta membingungkan kapan dan di mana alur kembali ke jalur utama. pada contoh ke 1 dan ke 2 seharusnya dilanjut dengan merge node agar Kembali ke jalur utama dan tidak menimbulkan kebingungan yang berkelanjutan

c. Menempatkan Banyak Aktivitas Secara Pararel Tanpa Fork Node

Contoh 1	Contoh 2
	

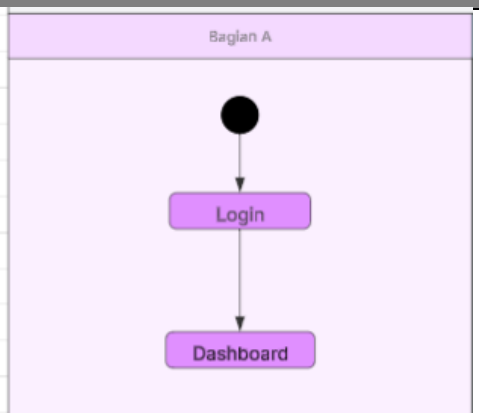
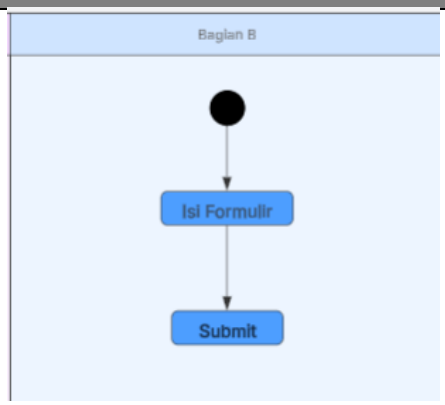
Tanpa fork node (garis horizontal dengan satu masuk dan banyak keluar), aktivitas paralel tidak tersampaikan dengan jelas. Pembaca bisa salah paham bahwa itu adalah urutan, bukan paralel.

d. Tidak Menunjukkan Kondisi Transisi Dengan Jelas

Contoh 1	Contoh 2
	

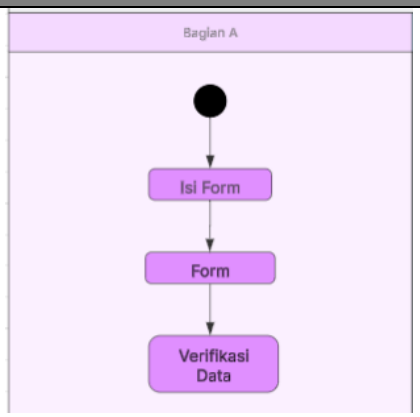
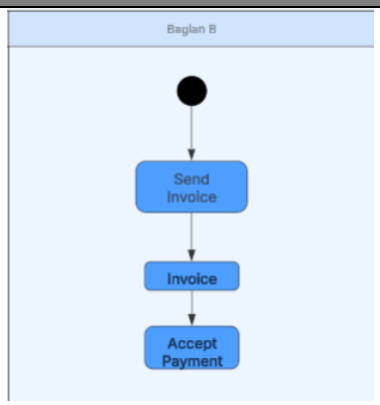
Transisi dari decision node harus diberi label kondisi (misal: [valid], [tidak valid]). Tanpa itu, diagram jadi tidak informatif dan keputusan tidak bisa dipahami secara logis.

e. Melupakan Alur Error atau Pengecualian

Contoh 1	Contoh 2
	

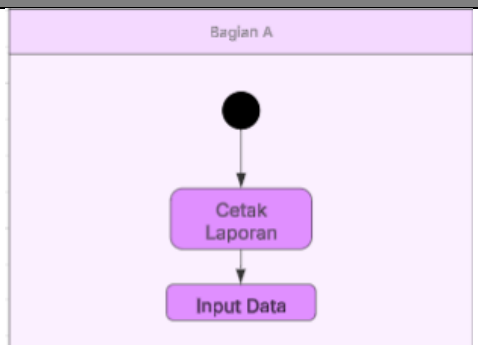
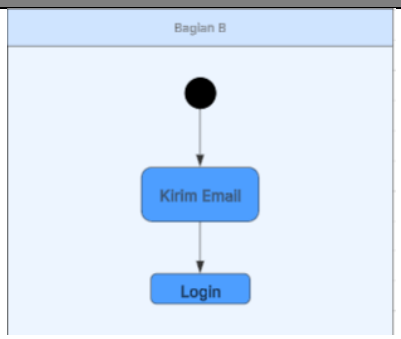
Tidak menyertakan error handling membuat diagram terlalu idealis dan tidak realistis. Sistem nyata selalu memiliki jalur untuk kondisi gagal.

f. Tidak Membedakan Antara Control Flow dan Object Flow

Contoh 1	Contoh 2
	

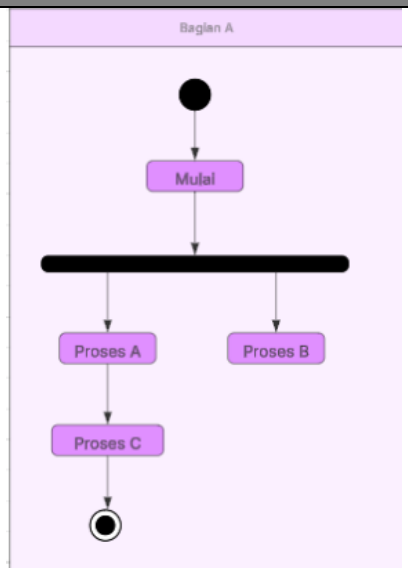
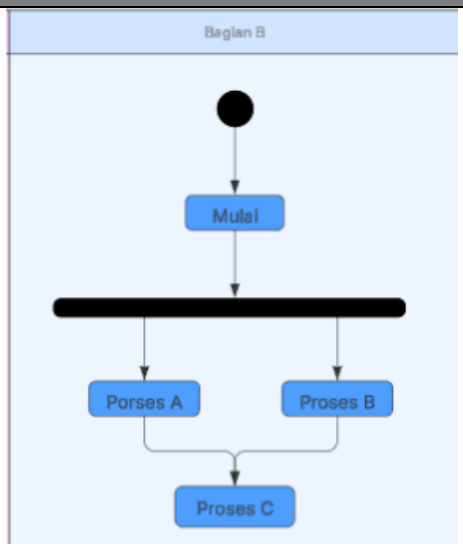
Harus dibedakan antara aktivitas (control flow) dan objek (object flow). Gunakan panah khusus dan object node untuk objek agar tidak tertukar sebagai aksi. Pada contoh harusnya untuk form dan invoice itu merupakan object sehingga panah yang digunakan bukan menggunakan panah untuk control flow, dan seharusnya menggunakan blok khusus object

g. Menampilkan Aktivitas tanpa Urutan Logis

Contoh 1	Contoh 2
	

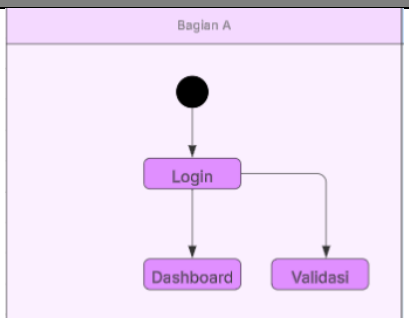
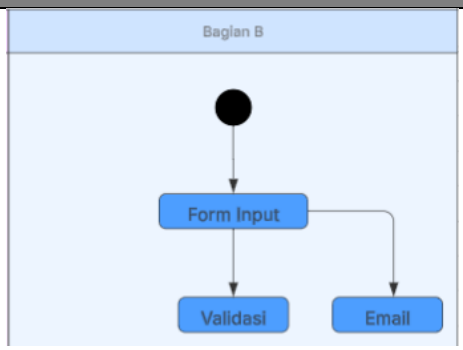
Urutan aktivitas harus mengikuti logika nyata. Menempatkan hasil akhir di awal membuat diagram tidak masuk akal dan menyesatkan pemahaman.

h. Tidak Menggambarkan Aktivitas Pararel Dengan Jelas

Contoh 1	Contoh 2
	

Harus ada join node untuk menyatukan kembali aktivitas paralel. Tanpa itu, proses lanjutan bisa dieksekusi sebelum semua aktivitas paralel selesai.

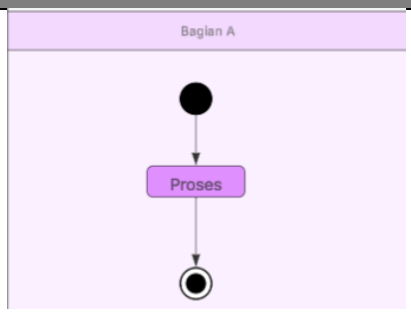
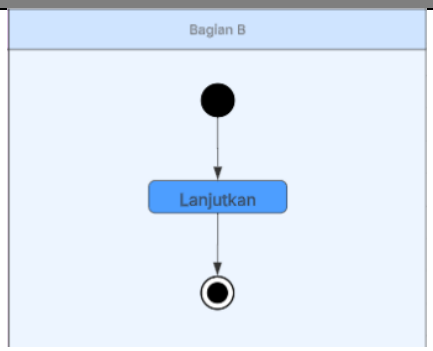
i. Membuat Panah Alur Silang Tanpa Alasan

Contoh 1	Contoh 2
	

Panah yang menyilang membuat diagram sulit dibaca. Sebaiknya tata ulang posisi elemen agar alur tetap rapi dan mengikuti arah vertikal atau horizontal.

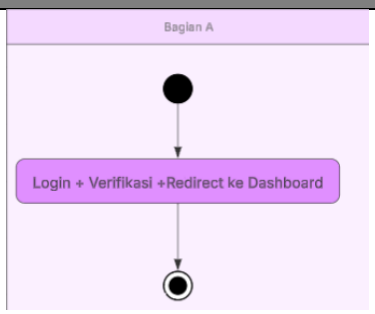
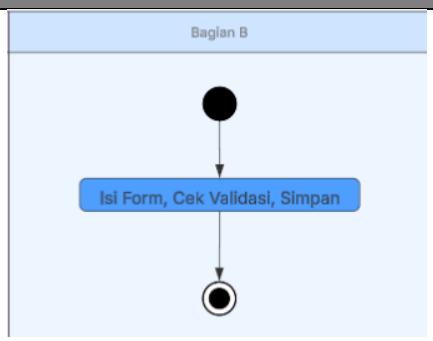
3.3.Kesalahan Representasi Aktivitas

a. Memberi label aktivitas yang tidak deskriptif

Contoh 1	Contoh 2
	

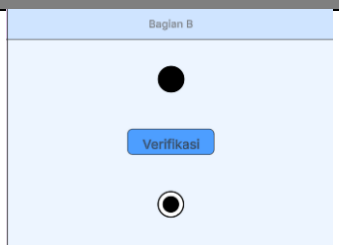
Nama aktivitas harus deskriptif agar pembaca tahu apa yang dilakukan. Hindari istilah generik seperti “proses”, “lanjut”, “kerjakan”, dll.

b. Menggabungkan terlalu banyak logika dalam satu aktivitas

Contoh 1	Contoh 2
	

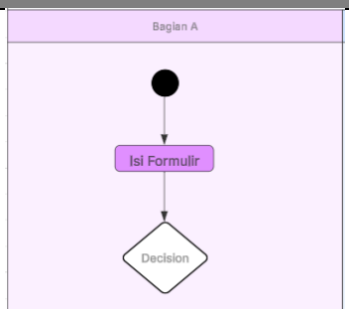
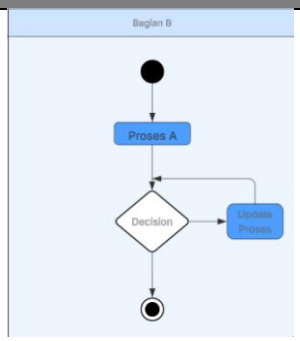
Satu aktivitas = satu tindakan tunggal. Gabungan logika menyulitkan analisis dan pemetaan alur kerja secara modular.

c. Menggambarkan aktivitas tanpa masukan atau keluaran

Contoh 1	Contoh 2
	

Setiap aktivitas harus berada dalam konteks aliran, baik sebagai hasil dari proses sebelumnya atau pengarah ke proses selanjutnya.

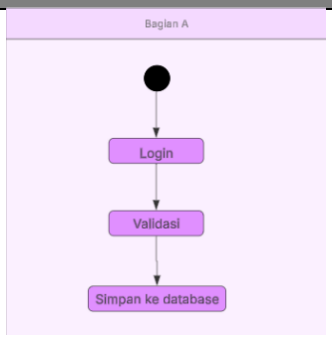
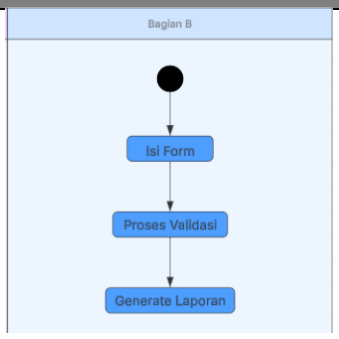
d. Menggabungkan flowchart tradisional dengan activity diagram

Contoh 1	Contoh 2
	

Activity diagram mengikuti notasi UML, bukan flowchart. Pencampuran simbol membuat diagram tidak standar dan sulit dimengerti oleh tim lain yang mengacu pada UML.

3.4.Kesalahan Terkait Swimlane dan Peran

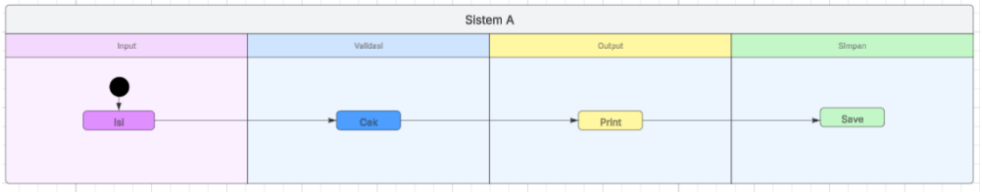
a. Tidak menggunakan swimlane untuk memisahkan peran

Contoh 1	Contoh 2
	

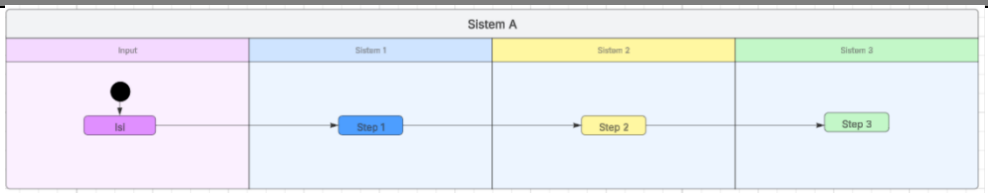
Swimlane penting untuk menunjukkan siapa melakukan apa. Tanpa itu, diagram tidak menunjukkan pembagian tanggung jawab, menyulitkan implementasi sistem

b. Membuat swimlane terlalu banyak tanpa pembenaran

Contoh 1



Contoh 2

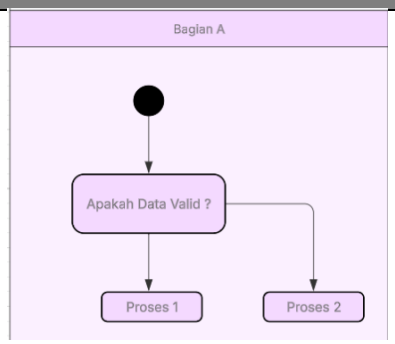


Gunakan swimlane hanya untuk memisahkan peran atau aktor, bukan untuk tahap proses atau komponen kecil dalam sistem. Terlalu banyak swimlane malah membuat diagram tidak terbaca.

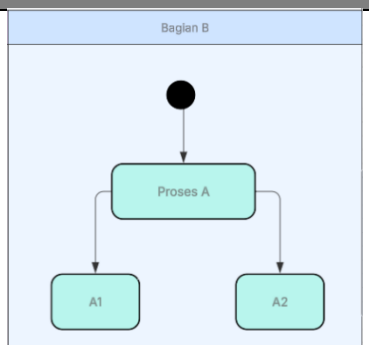
3.5.Kesalahan Notasi dan Standarisasi

a. Tidak menggunakan notation yang standar

Contoh 1



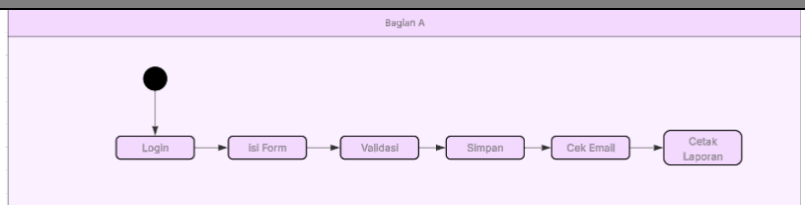
Contoh 2



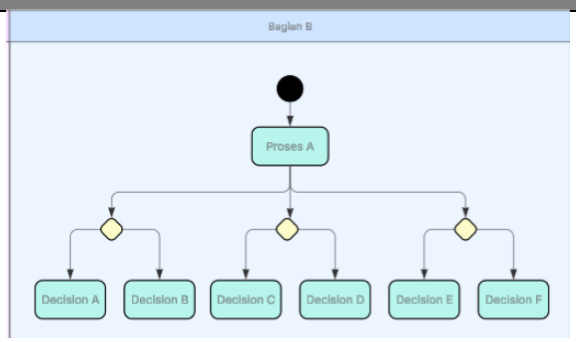
Menggunakan notasi non-standar menyebabkan diagram tidak interoperabel dan membingungkan orang lain yang terbiasa membaca UML. Standar simbol UML harus diikuti agar komunikatif dan universal.

b. Menggambar diagram terlalu kompleks

Contoh 1

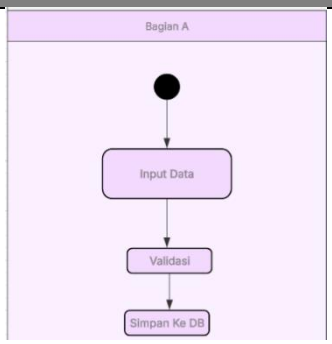
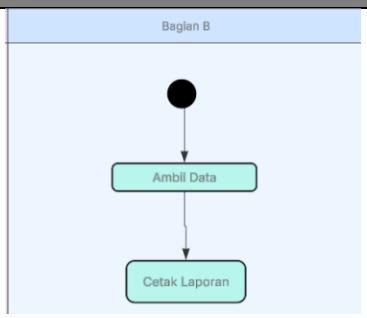


Contoh 2



Activity diagram yang terlalu kompleks justru melawan tujuannya: membuat proses mudah dipahami. Pecah alur besar menjadi bagian-bagian kecil yang lebih fokus.

c. Tidak menyertakan objek atau data penting

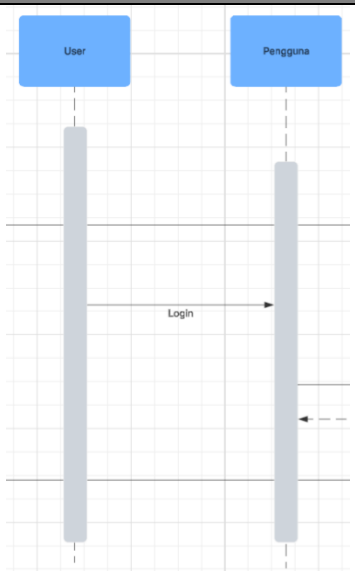
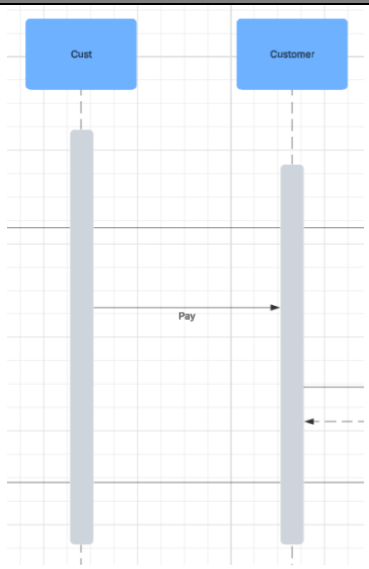
Contoh 1	Contoh 2
	

Objek (data, dokumen, file) yang digunakan atau dihasilkan harus ditampilkan sebagai object node (biasanya dibedakan dengan label <<object>>). Ini memperjelas apa yang diproses oleh sistem.

4. KESALAHAN UMUM DALAM PEMBUATAN SEQUENCE DIAGRAM

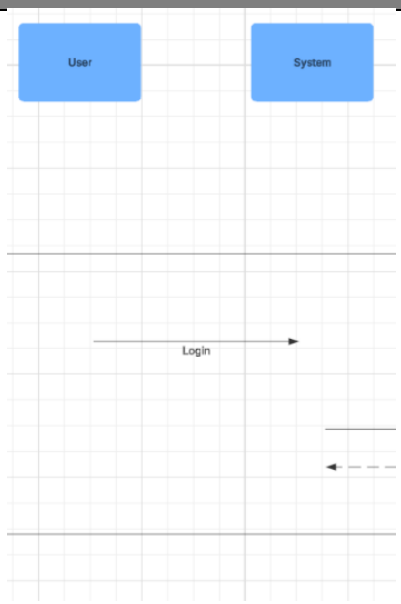
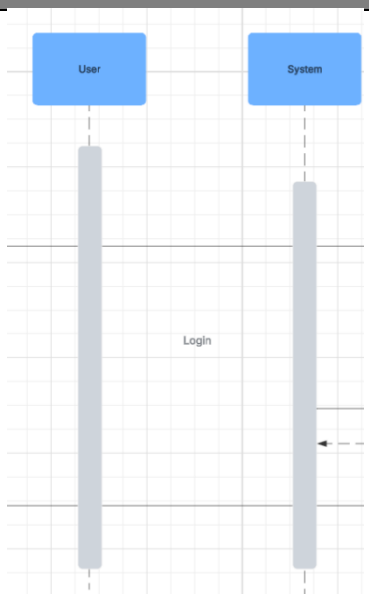
4.1. Kesalahan Representasi Objek

a. Menggunakan nama objek yang tidak konsisten

Contoh 1	Contoh 2
	

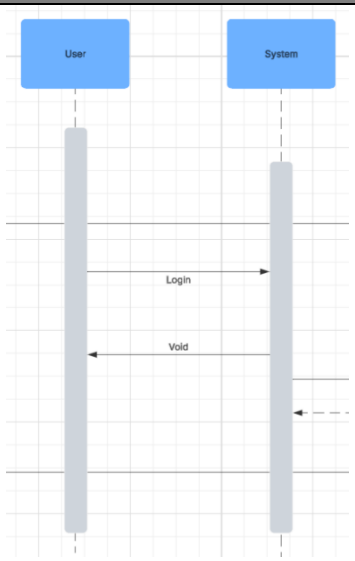
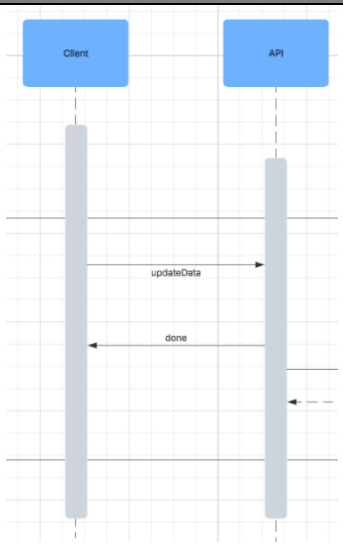
Nama objek harus konsisten dengan class diagram agar keterkaitan antar diagram tetap jelas. Nama seperti :Cust dan :Customer bisa menimbulkan ambiguitas, terutama pada proyek besar dengan banyak pengembang.

b. Tidak menunjukkan waktu hidup (lifeline) objek

Contoh 1	Contoh 2
	

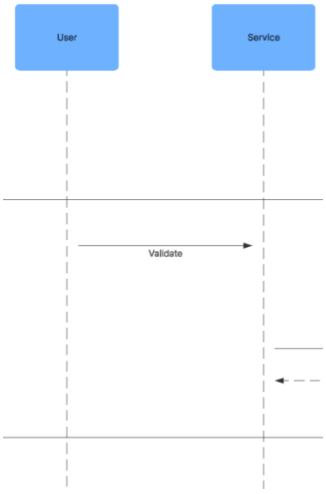
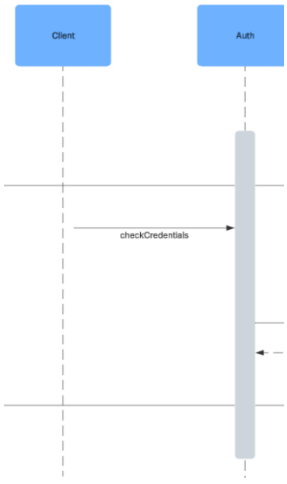
Lifeline diperlukan untuk menunjukkan kapan objek hidup dan berinteraksi dalam skenario. Tanpa lifeline, interaksi tampak terputus dan membingungkan.

c. Menggambarkan return message yang tidak perlu

Contoh 1	Contoh 2
	

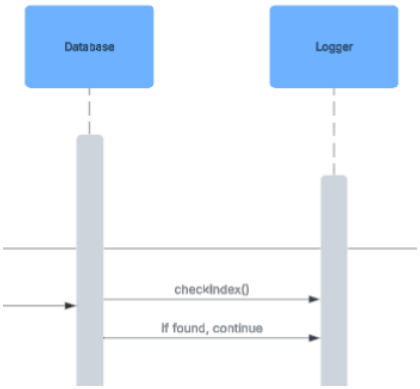
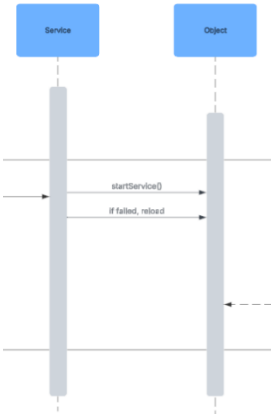
Return message digunakan bila hasilnya dibutuhkan untuk proses selanjutnya. Jika tidak, cukup tampilkan call message untuk menjaga diagram tetap bersih.

d. Mengabaikan penggunaan activation bar

Contoh 1	Contoh 2
	

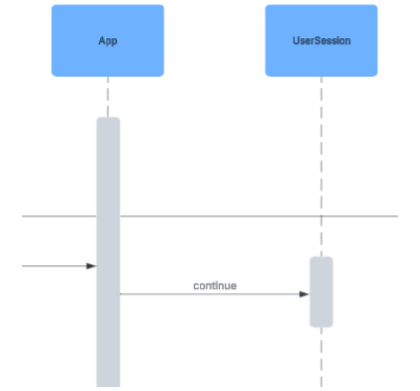
Activation bar menunjukkan kapan objek sedang aktif menjalankan proses. Tanpanya, diagram tidak menunjukkan durasi pemrosesan secara visual.

e. Menambahkan proses internal objek dalam diagram

Contoh 1	Contoh 2
	

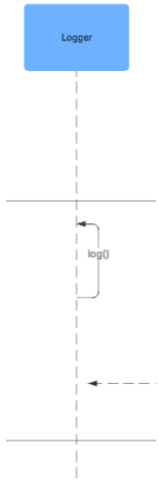
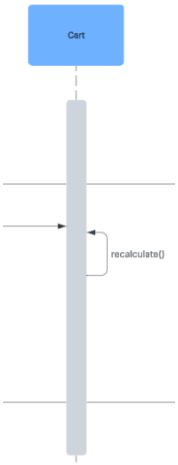
Sequence diagram memvisualkan interaksi antar objek, bukan detail internal logika. Logika internal tempatnya di activity diagram atau dalam dokumentasi kode.

f. Tidak menunjukkan inisialisasi objek

Contoh 1	Contoh 2
	

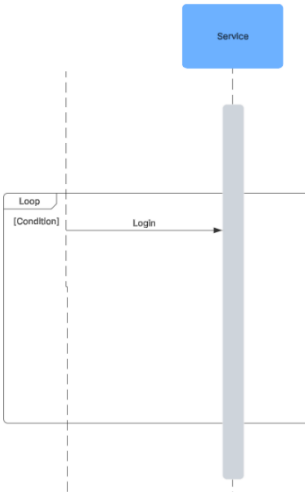
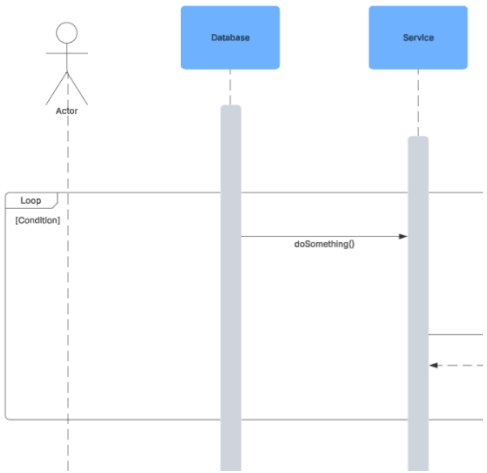
Objek yang dibuat saat runtime harus ditampilkan dengan create() message dari objek pembuat agar alur pembentukan objek bisa ditelusuri.

g. Objek mengirim pesan kepada dirinya sendiri tanpa kejelasan

Contoh 1	Contoh 2
	

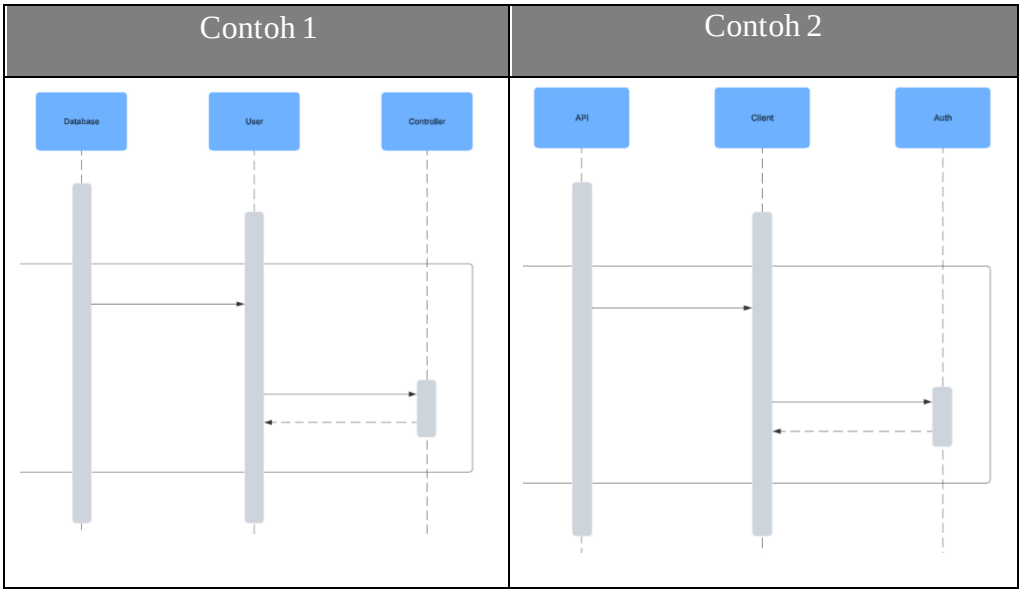
Self-call sah-sah saja, tapi harus digambarkan dengan jelas dan hanya jika diperlukan. Jangan sampai malah membingungkan.

h. Tidak menyertakan aktor eksternal

Contoh 1	Contoh 2
	

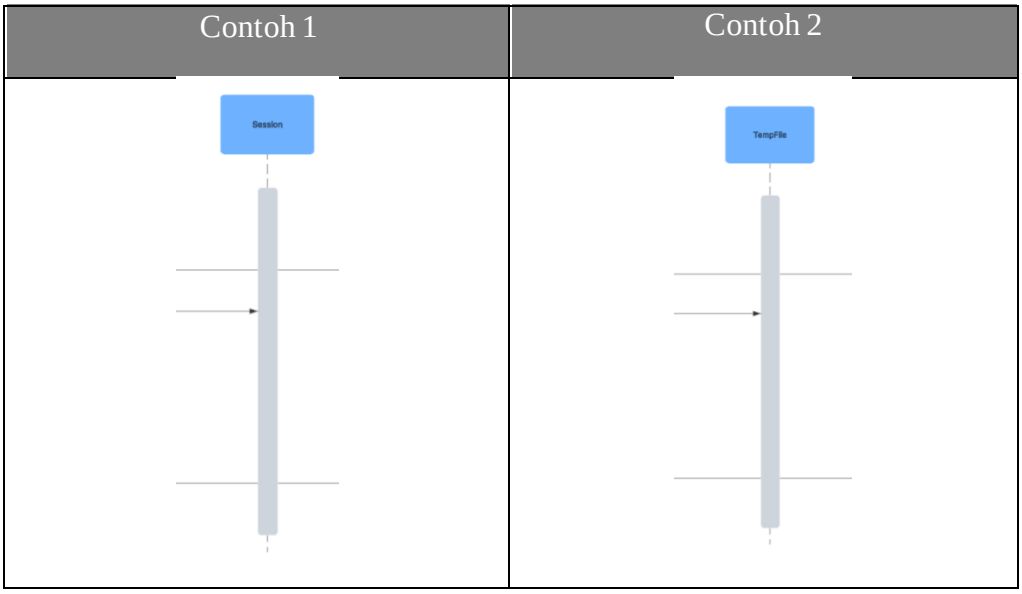
Sequence diagram harus menunjukkan dari mana skenario dimulai — biasanya oleh aktor eksternal (user, sistem lain, API).

i. Menempatkan objek dalam urutan horizontal yang membingungkan



Urutan objek idealnya dari kiri ke kanan mengikuti alur interaksi. Ini membuat pembaca lebih mudah mengikuti logika komunikasi.

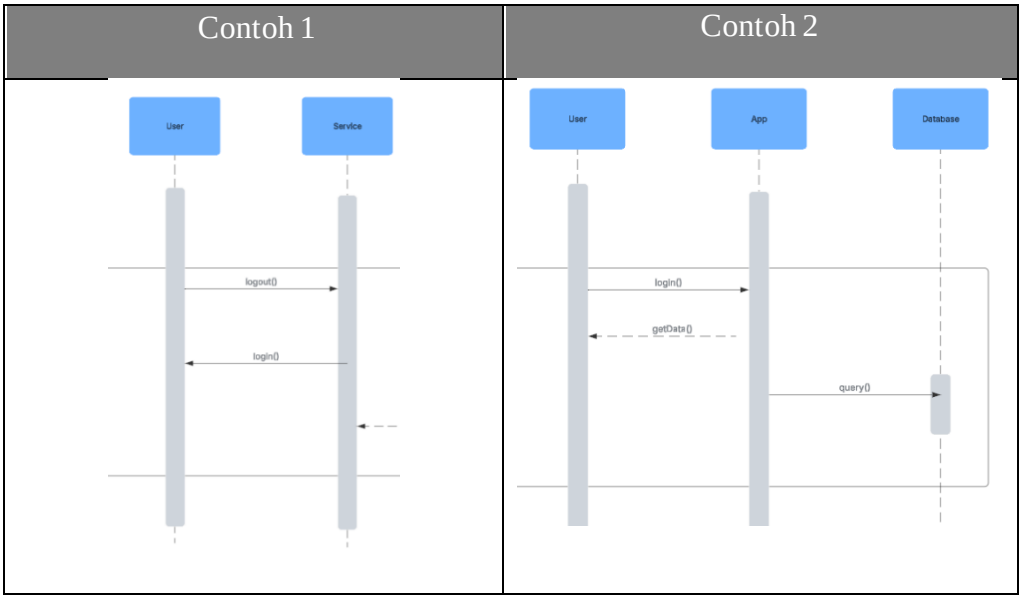
j. Tidak menggambarkan penghapusan objek jika relevan



Jika objek di-terminate dalam skenario, harus digambarkan dengan simbol X pada lifeline. Ini penting untuk menandai akhir siklus hidup objek.

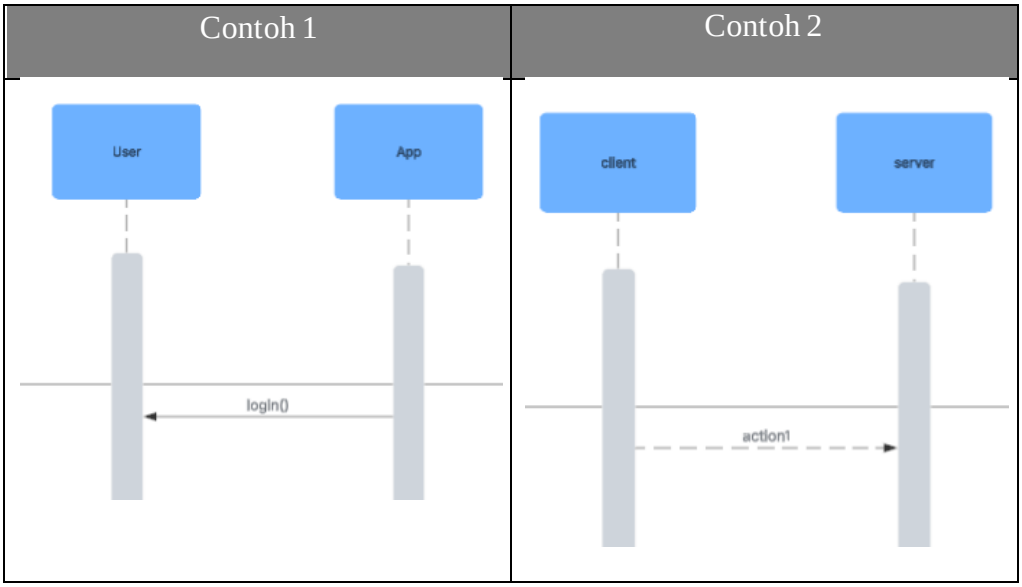
4.2. Kesalahan Dalam Penggambaran Pesan

a. Tidak menunjukkan urutan pesan secara benar



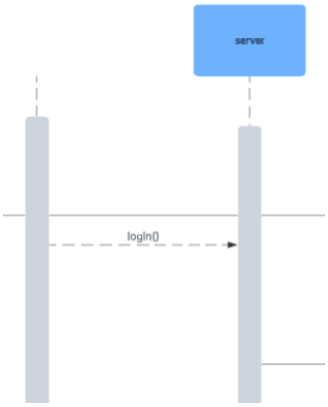
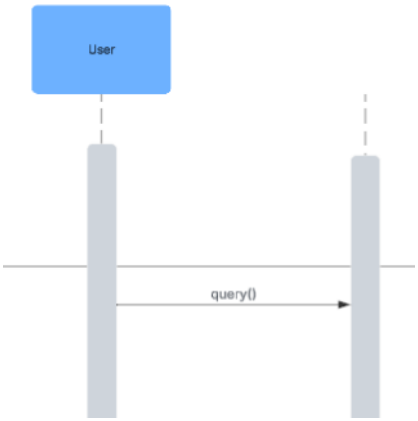
Sequence diagram disusun dari atas ke bawah sesuai urutan waktu. Pesan harus digambar berurutan, bukan acak.

b. Menggunakan panah arah yang salah untuk pesan



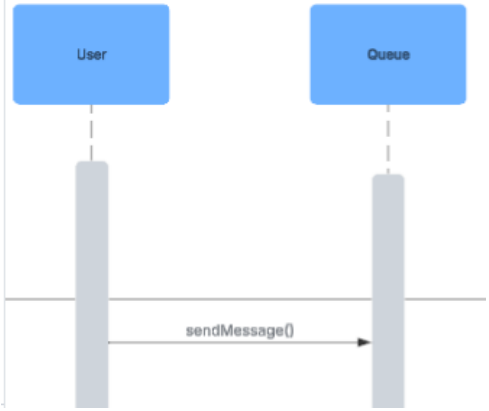
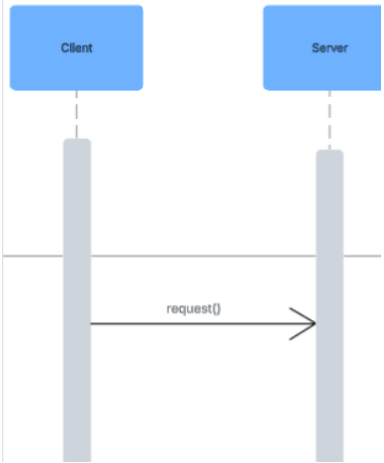
Menggunakan arah atau jenis panah yang salah bisa mengubah makna komunikasi dan menyesatkan pembaca.

c. Tidak menyertakan objek pengirim atau penerima pesan

Contoh 1	Contoh 2
	

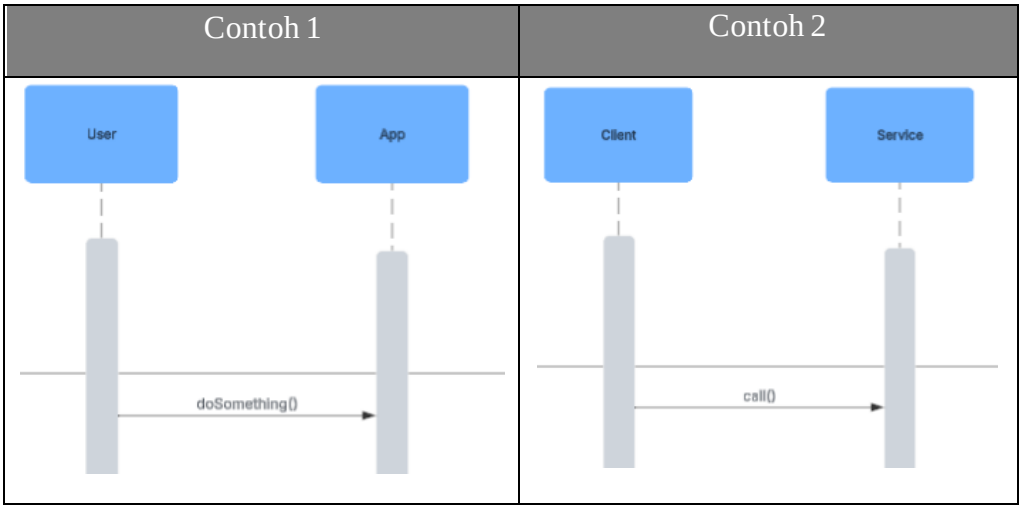
Setiap pesan harus punya pengirim dan penerima agar alur komunikasi bisa ditelusuri secara utuh.

d. Tidak membedakan synchronous dan asynchronous message

Contoh 1	Contoh 2
	

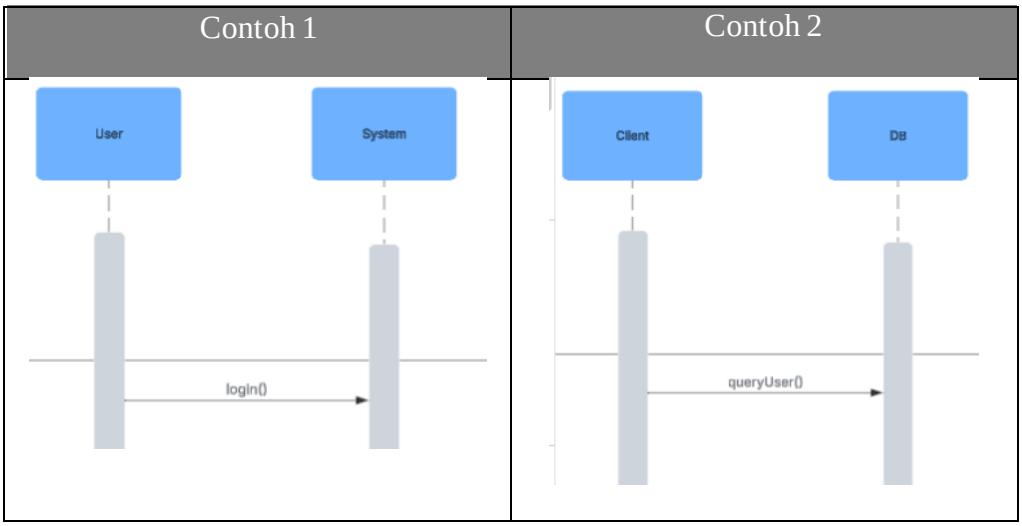
Jenis pesan memengaruhi bagaimana proses dijalankan (blocking atau tidak). Gunakan jenis panah yang tepat sesuai sifat komunikasi.

e. Tidak mencantumkan pesan eksplisit saat memanggil metode



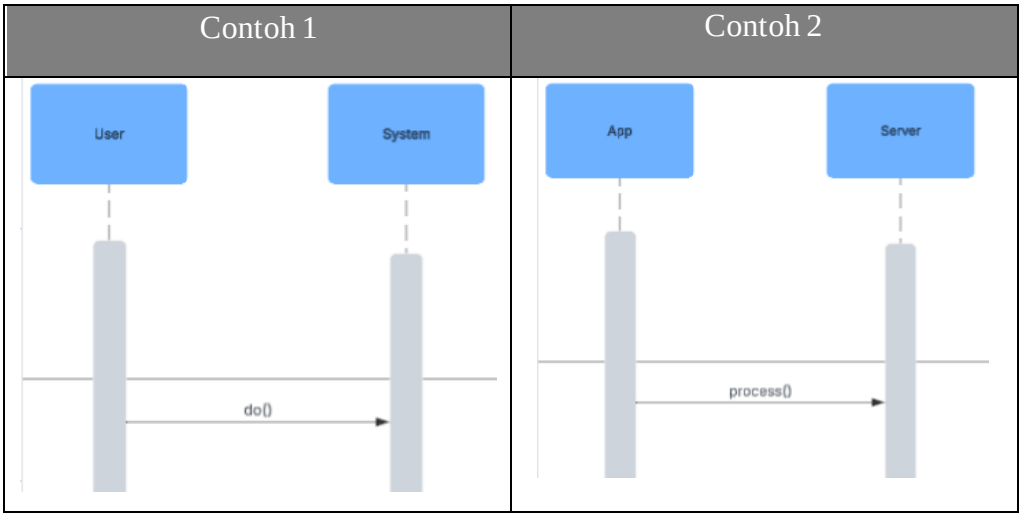
Nama pesan harus menjelaskan metode yang dipanggil secara jelas, agar pembaca tahu tujuan interaksi.

f. Tidak menggambarkan hasil dari operasi penting



Jika hasil dari pesan digunakan untuk proses berikutnya, wajib ditampilkan sebagai return message.

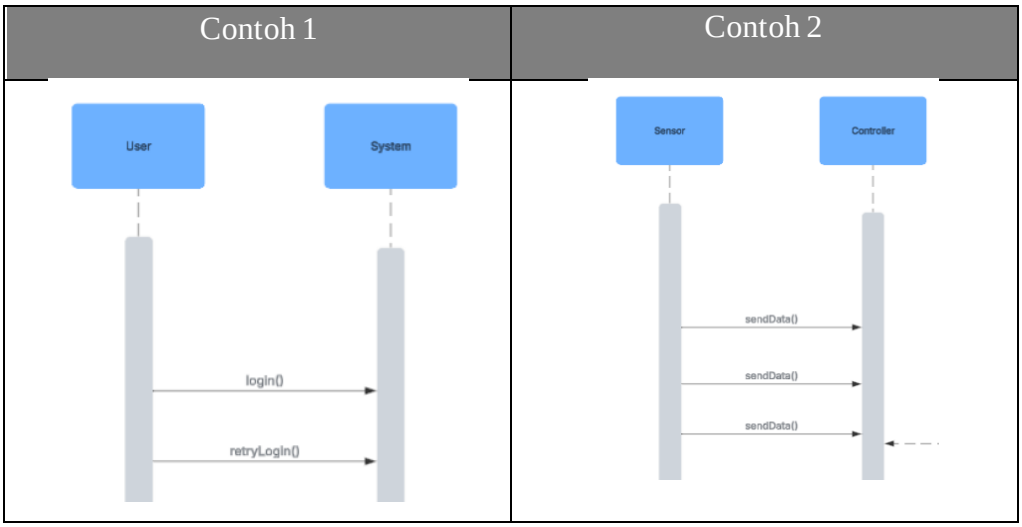
g. Tidak memberikan nama yang bermakna untuk pesan



Pesan harus menjelaskan apa yang dilakukan, mirip seperti nama fungsi. Hindari nama umum dan tidak informatif.

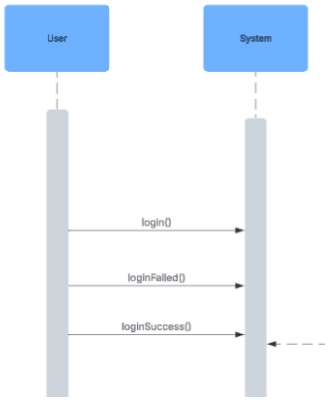
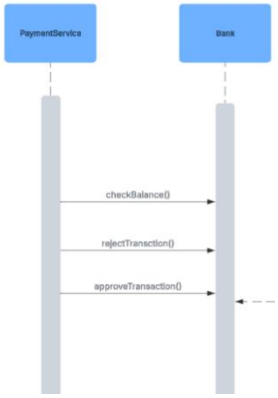
4.3. Kesalahan Penggunaan Fragment

a. Melupakan penggambaran kondisi atau loop



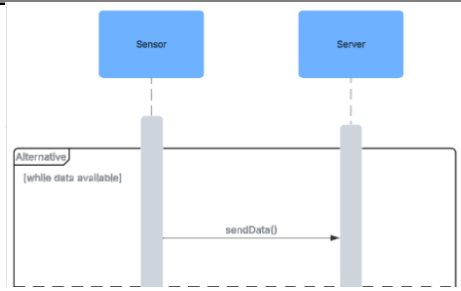
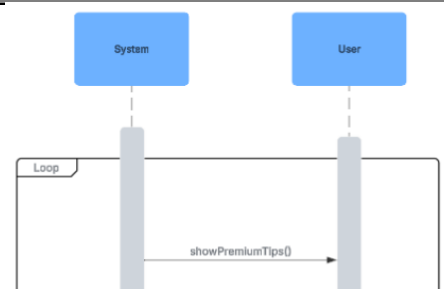
Jika ada proses berulang atau percobaan ulang, kamu wajib gunakan loop fragment agar alur tidak ambigu dan struktur kontrol logika terlihat jelas

b. Mencampurkan alur logika alternatif tanpa pengelompokan

Contoh 1	Contoh 2
	

Alternatif kondisi seperti if-else harus dibungkus dalam alt fragment. Tanpanya, urutan skenario jadi ambigu, seolah semua cabang terjadi bersamaan.

c. Kesalahan penggunaan diagram fragment (alt, loop, opt)

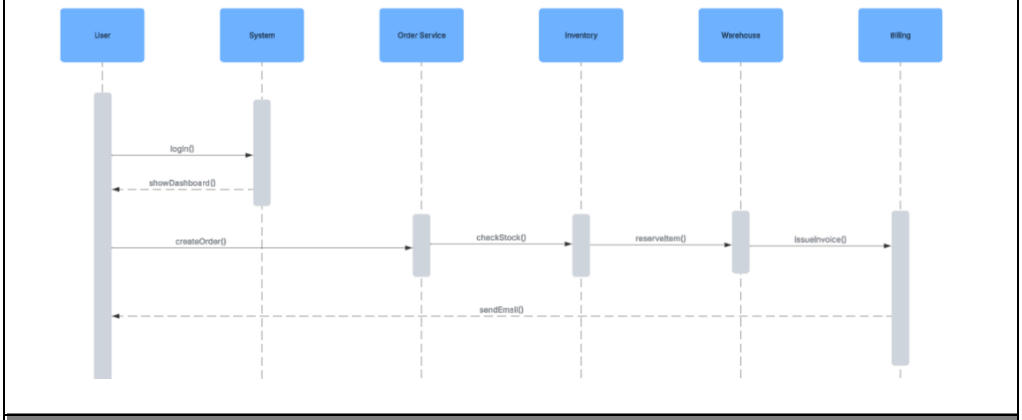
Contoh 1	Contoh 2
	

Kesalahan penggunaan fragment terjadi ketika jenis struktur kontrol seperti alt, loop, atau opt digunakan secara tidak tepat atau tidak dilengkapi dengan label kondisi. Fragment alt seharusnya hanya digunakan untuk menggambarkan cabang logika (seperti if/else), bukan untuk pengulangan, sementara loop digunakan untuk menyatakan proses berulang, dan opt digunakan untuk kondisi opsional tunggal. Tanpa kondisi eksplisit pada fragment seperti opt, diagram menjadi ambigu karena pembaca tidak tahu kapan bagian tersebut seharusnya dijalankan. Kesalahan seperti ini membuat struktur kontrol dalam skenario tidak terbaca dengan baik, mengurangi kejelasan logika sistem, dan menyulitkan pengembang atau analis dalam memahami perilaku yang sebenarnya.

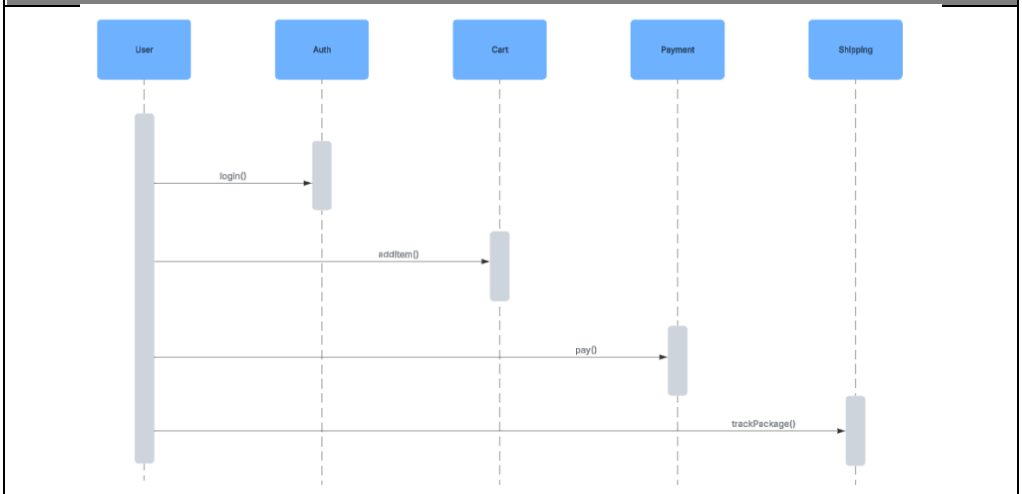
4.4. Kesalahan Fokus dan Ruangan Lingkup Diagram

a. Terlalu banyak logika dalam satu diagram

Contoh 1

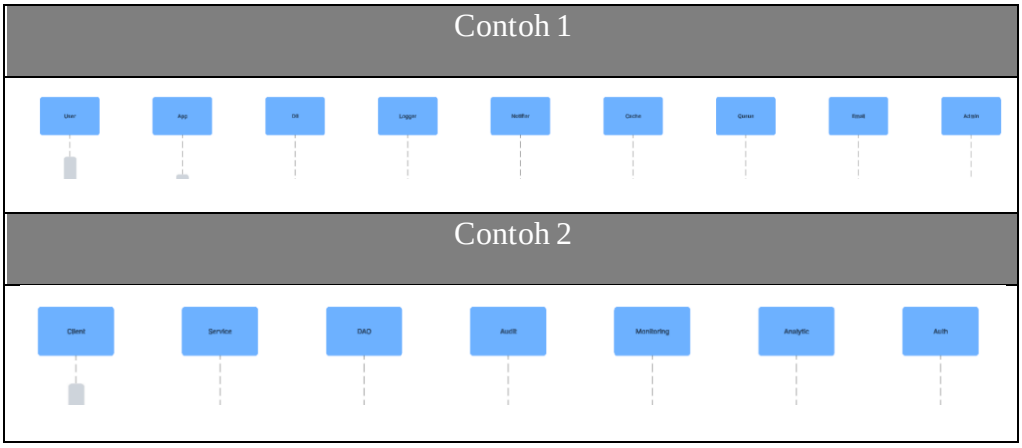


Contoh 2



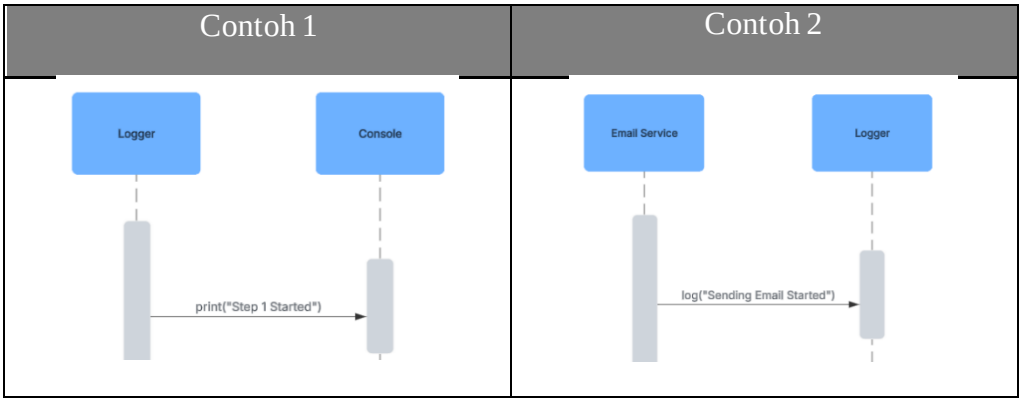
Sequence diagram idealnya hanya menggambarkan satu skenario utama per diagram. Jika terlalu banyak logika dan proses dimasukkan sekaligus, diagram menjadi panjang, penuh, dan sulit dibaca. Hal ini membuat pemisahan konteks dan pelacakan skenario menjadi tidak efisien. Solusinya adalah memecah skenario menjadi beberapa diagram berdasarkan alur seperti login, order, dan pembayaran.

b. Terlalu banyak objek dalam satu diagram



Terlalu banyak objek akan membuat sequence diagram tampak padat horizontal, menyulitkan pembacaan alur dan hubungan antar objek. Hal ini juga mengaburkan skenario utama karena perhatian pembaca terbagi. Solusinya: gunakan diagram terpisah untuk proses berbeda atau kelompokkan logika interaksi dalam domain yang sama saja.

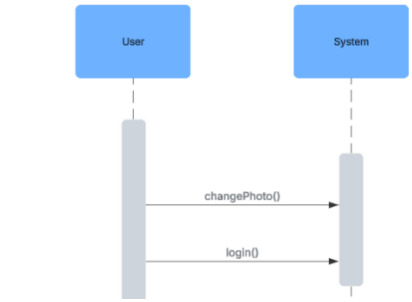
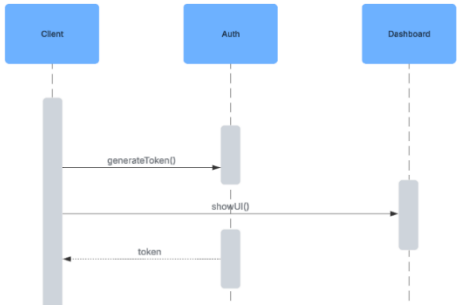
c. Menggambar interaksi yang tidak relevan



Hanya interaksi yang berkontribusi langsung terhadap skenario utama yang seharusnya digambarkan. Menambahkan logging, debugging, atau proses background seperti monitoring hanya akan menambah beban visual tanpa memperjelas proses bisnis. Sequence diagram bukan tempat untuk mencatat aktivitas pendukung diagram menjadi informatif jika isinya fokus.

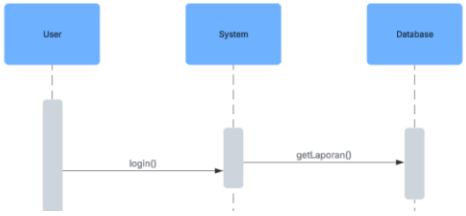
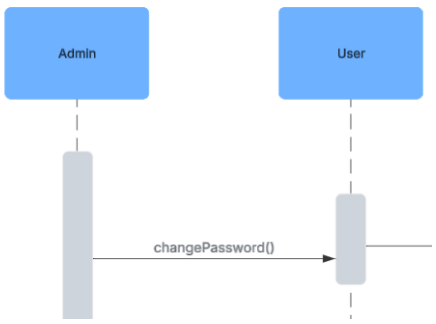
4.5. Ketidaksesuaian Dengan Diagram Lain / UML

a. Tidak memperlihatkan urutan waktu yang logis

Contoh 1	Contoh 2
	

Sequence diagram menyajikan komunikasi antar objek secara berurutan dari atas ke bawah sesuai waktu nyata. Jika urutan pesan tidak sesuai logika waktu atau proses, maka diagram akan membingungkan dan menyebabkan salah interpretasi terhadap skenario sistem. Ini bisa menimbulkan bug saat pengembangan karena urutan eksekusi salah ditafsirkan.

b. Tidak konsisten dengan usecase diagram

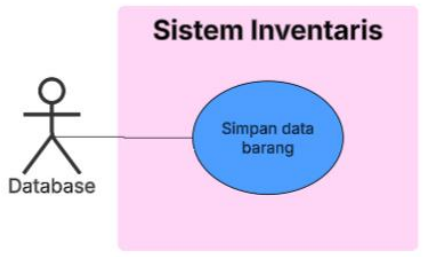
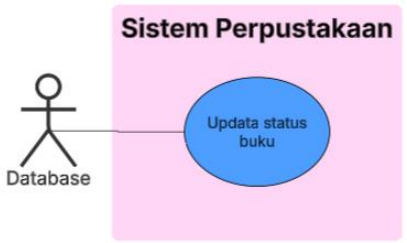
Contoh 1	Contoh 2
Dalam usecase seharusnya : User melakukan login → sistem validasi → user melihat dashboard 	Dalam usecase seharusnya : Admin menghapus akun user 

Sequence diagram harus menjadi representasi langsung dari satu skenario dalam use case diagram. Jika sequence diagram menggambarkan aktivitas di luar use case yang dirujuk, maka ada ketidaksesuaian yang bisa membingungkan stakeholder. Ini juga akan menyulitkan validasi sistem karena alur yang digambarkan tidak sesuai kebutuhan yang didefinisikan.

5. KESALAHAN UMUM DALAM PEMBUATAN USECASE DIAGRAM

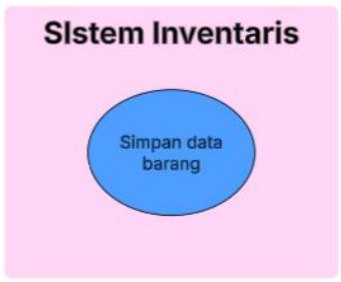
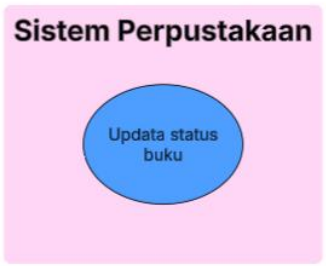
5.1. Kesalahan Representasi Aktor

a. Menyertakan database sebagai aktor

Contoh 1	Contoh 2
	

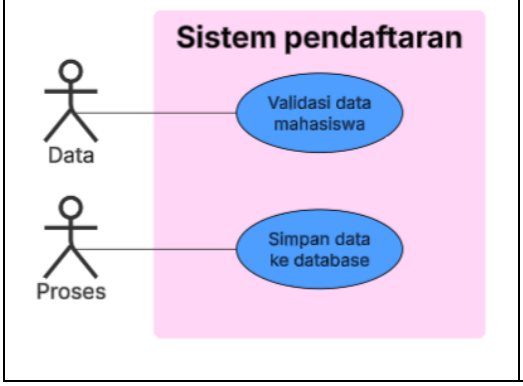
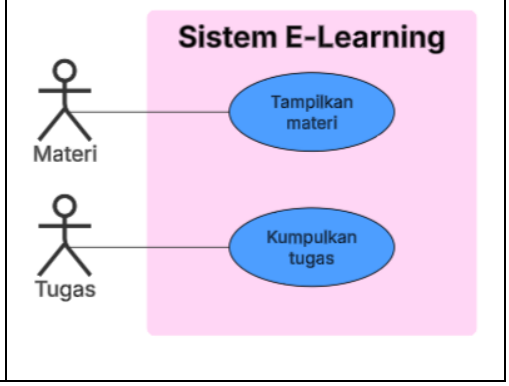
Database dianggap sebagai aktor, padahal itu bagian internal sistem. Seharusnya, adalah manusia (user) atau sistem eksternal. Database hanya digunakan *di balik layar* dalam sistem—tidak langsung berinteraksi.

b. Menggambarkan usecase tanpa aktor

Contoh 1	Contoh 2
	

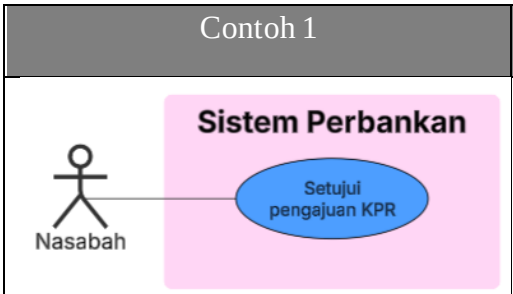
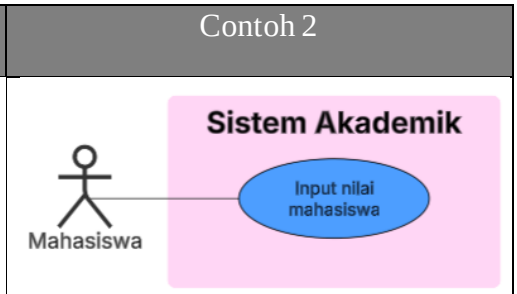
Pada usecase diagram, sebuah usecase haruslah memiliki aktor yang akan memicu usecase yang bersangkutan itu aktif.

c. Terlalu banyak aktor fiktif atau abstrak

Contoh 1	Contoh 2
	

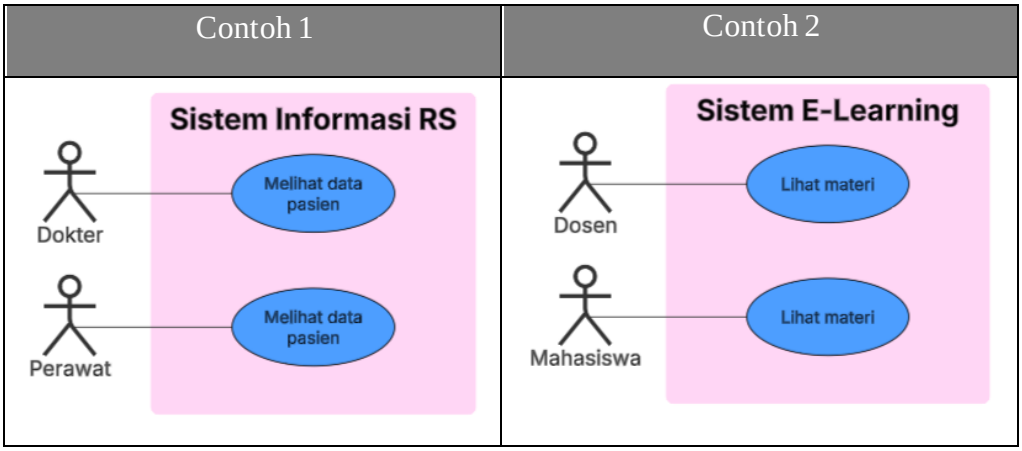
menambahkan terlalu banyak aktor yang tidak relevan, fiktif, atau terlalu abstrak (seperti "Proses", "Data", "Sistem Internal", dll), padahal aktor seharusnya entitas luar yang berinteraksi langsung dengan sistem.

d. Menghubungkan aktor dengan usecase yang tidak relevan

Contoh 1	Contoh 2
	

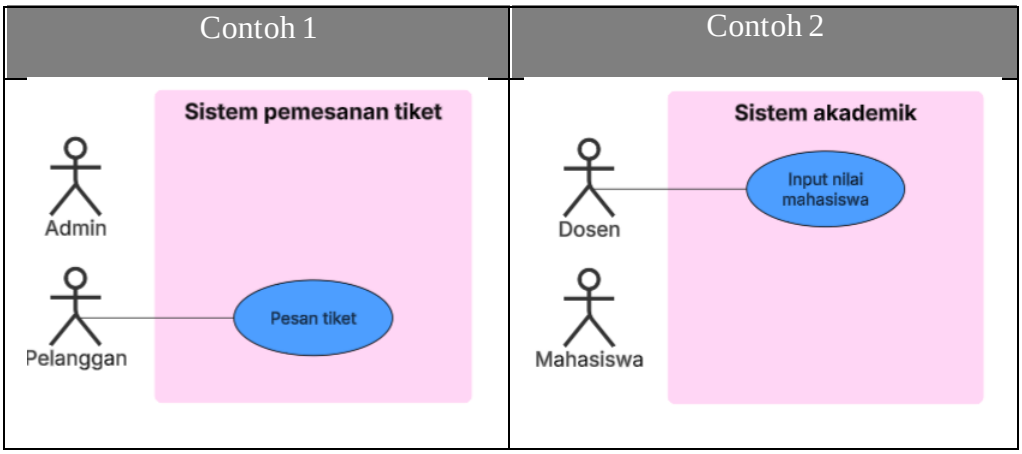
Aktor pada usecase diagram harus memiliki hubungan dengan usecase yang relevan. Pada sistem perbankan, nasabah seharusnya tidak bisa menyetujui pengajuan KPR, melaikan petugas bank-nya lah yang bisa. Pada sistem akademik juga mahasiswa tentunya tidak bisa menginputkan nilai mahasiswa, yang mana biasanya dilakukan oleh dosen atau petugas akademik.

e. Mengabaikan relasi umum antar aktor



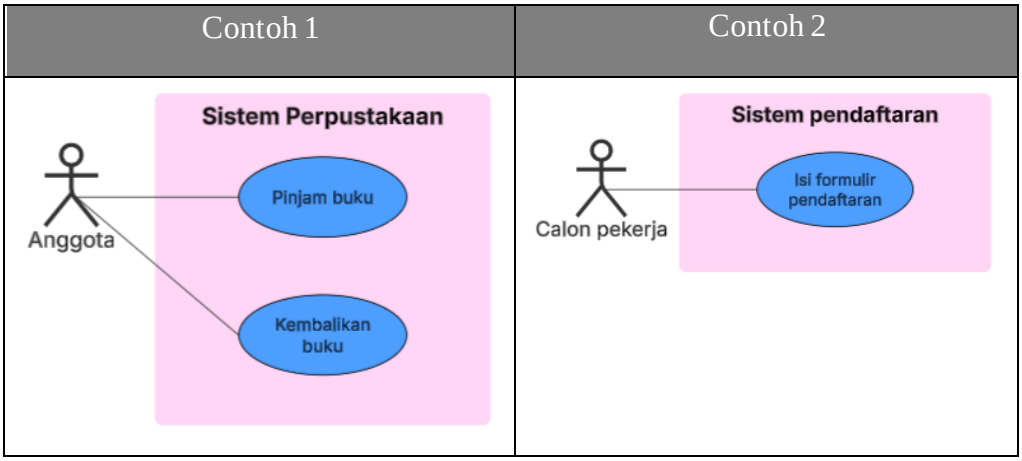
Dokter dan perawat memiliki perilaku yang sama, begitu juga dosen dan mahasiswa. Ini merupakan duplikasi. Jika ada aktor yang memiliki perilaku yang sama, maka harus dilakukan generalisasi agar tidak ada duplikasi use case.

f. Membuat aktor tanpa tujuan atau fungsionalitas



Aktor harus mewakili entitas yang berinteraksi langsung dengan sistem melalui satu atau lebih use case. Jika aktor tidak terhubung ke satu pun use case, maka dia tidak memiliki peran nyata dan tidak perlu digambar.

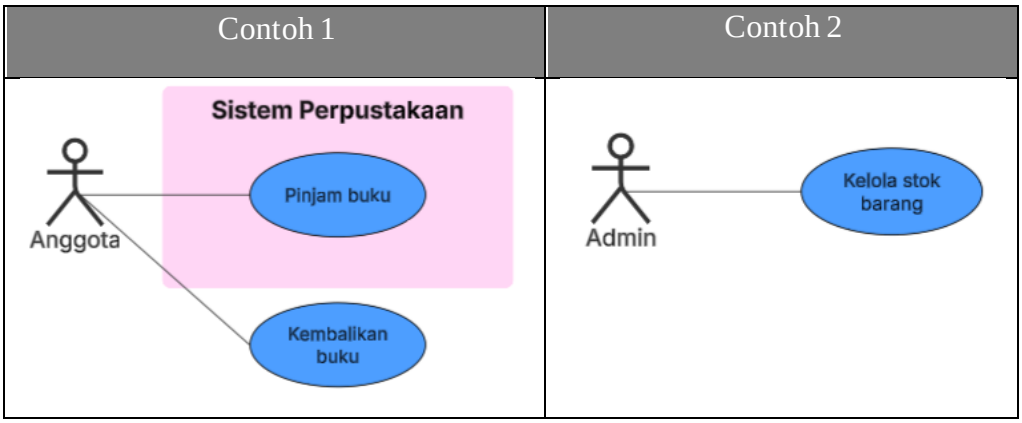
g. Tidak mengidentifikasi semua aktor yang relevan



Beberapa aktor penting tidak dimasukkan, meskipun mereka berinteraksi langsung dengan sistem atau memiliki kepentingan terhadap use case tertentu. Pada contoh 1, seharusnya ada petugas perpustakaan yang mengelola data buku dan cek laporan peminjaman. Pada contoh 2, seharusnya ada panitia seleksi yang melakukan seleksi kepada calon pekerja.

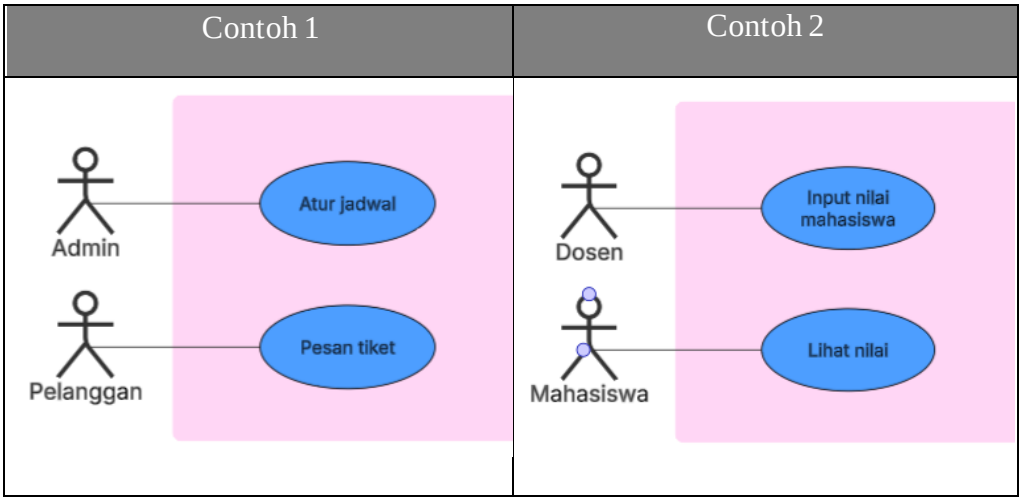
5.2. Kesalahan pada Struktur dan Elemen Boundary

a. Menempatkan sistem di luar batas sistem (system boundary)



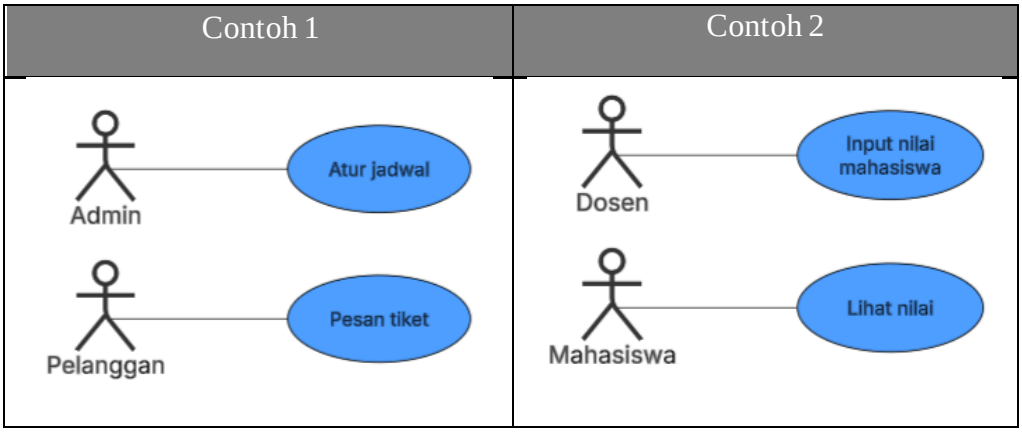
Use case diagram harus memiliki "system boundary box" yang menandakan sistem yang sedang dianalisis. Sistem harus berada di dalam kotak, bukan sebagai aktor eksternal. Kalau sistem ditempatkan sebagai aktor, itu artinya interaksi sistem dimodelkan dengan dirinya sendiri, yang tidak masuk akal.

b. Tidak menyertakan nama sistem pada boundary



Kotak boundary (persegi panjang besar yang membungkus semua use case) harus selalu diberi nama sistem yang sedang dimodelkan. Ini penting agar pembaca tahu sistem apa yang sedang dianalisis, terutama bila terdapat banyak sistem berbeda dalam satu proyek.

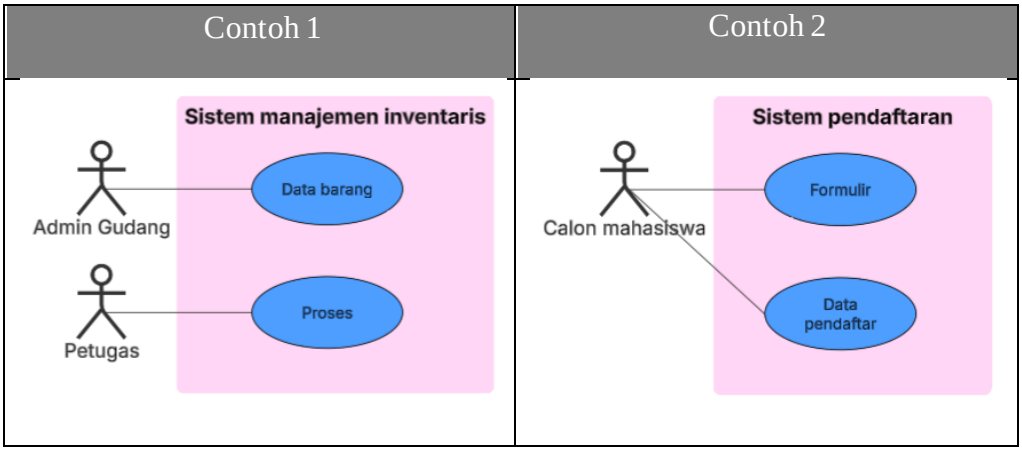
c. Membuat usecase diagram tampa batas sistem



Batas sistem (system boundary) adalah persegi panjang besar yang menampung semua use case. Tanpa kotak ini, tidak jelas mana yang termasuk bagian sistem dan mana yang bukan. Ini bisa membingungkan dan menghilangkan konteks utama dari use case diagram.

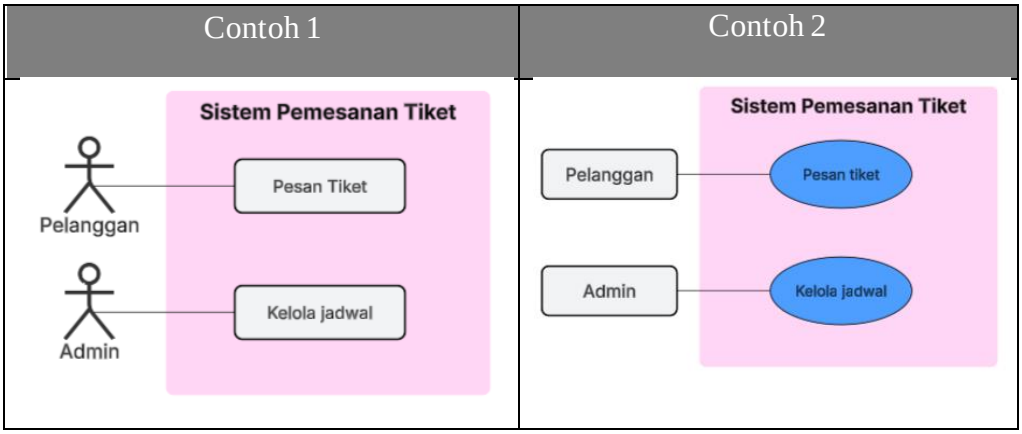
5.3. Kesalahan Penamaan dan Notasi

a. Tidak menggunakan nama yang representatif untuk usecase



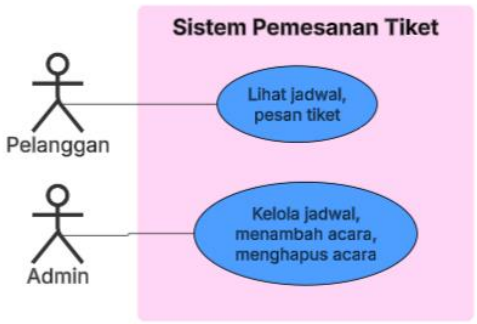
Use case harus menggambarkan tujuan pengguna (aktor) dalam bentuk kata kerja aktif, seperti "Ajukan Pendaftaran", "Lihat Transkrip", atau "Bayar Tagihan". Kesalahan umum yang biasanya terjadi adalah menggunakan nama yang terlalu umum dan tidak menggambarkan tindakan.

b. Menggunakan symbol yang tidak standar UML



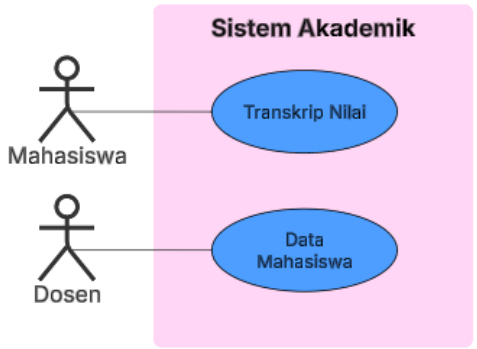
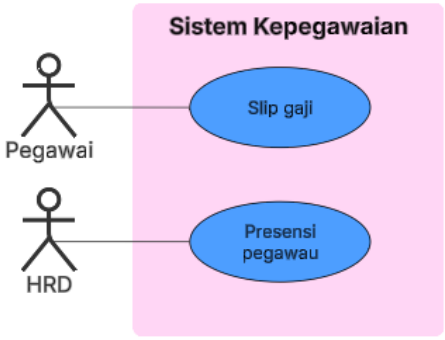
Gunakan hanya simbol yang sesuai standar UML. Contohnya, usecase seharusnya oval, bukan kotak dan aktor seharusnya stickman atau icon manusia, bukan kotak.

c. Menyatukan usecase berbeda dalam satu symbol

Contoh 1	Contoh 2
	

Use case seharusnya fokus pada satu fungsi tunggal dan spesifik dan tidak menyatukan banyak proses atau aksi ke dalam satu oval. Kalau disatukan, diagram jadi tidak modular, sulit dibaca, dan sulit dipelihara.

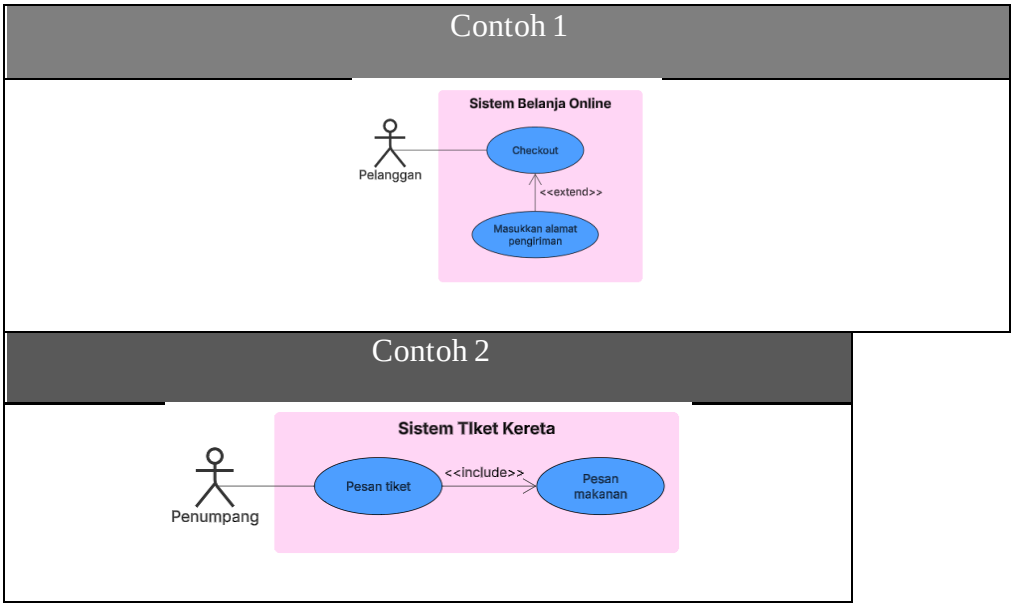
d. Menggunakan kata benda untuk nama usecase

Contoh 1	Contoh 2
	

Nama use case seharusnya menggunakan kata kerja aktif, karena use case mewakili aksi atau tujuan dari aktor terhadap sistem. Jika menggunakan kata benda, maka akan yidak jelas apa yang dilakukan oleh aktor. Juga use case tampak seperti entitas/data, bukan proses interaktif

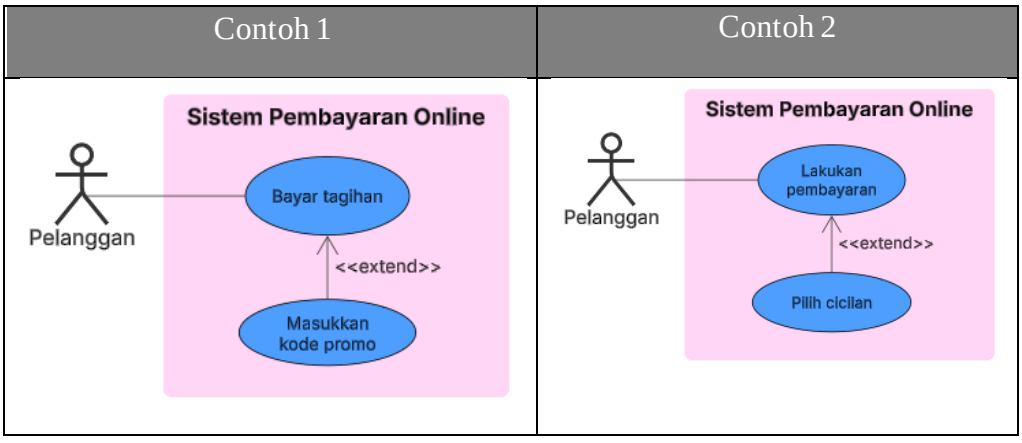
5.4. Kesalahan Relasi Use Case

a. Salah menggunakan relasi <<include>> dan <<extend>>



Relasi include digunakan jika use case A selalu menyertakan use case B (Wajib). Relasi extend digunakan jika use case B bersifat opsional, hanya terjadi di kondisi tertentu use case A. Pada sistem belanja online, memasukkan alamat itu merupakan sesuatu yang wajib, sehingga harusnya digunakan include. Pada sistem tiket kereta, pemesanan makanan itu bersifat opsional, sehingga harusnya digunakan extend.

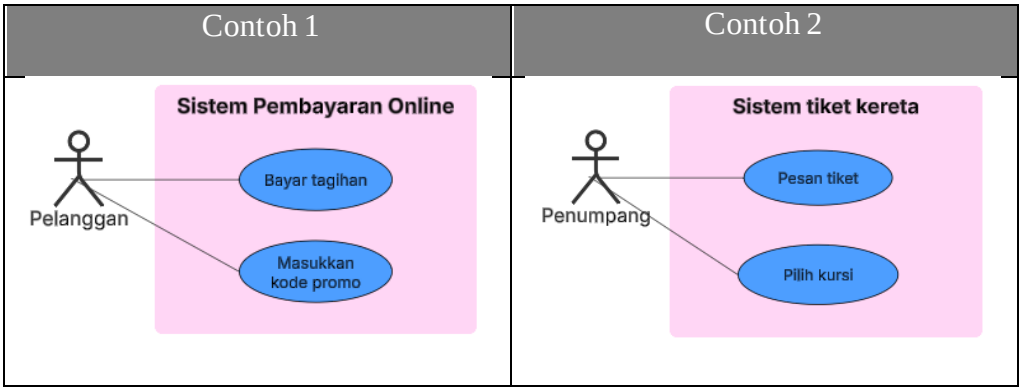
b. Tidak menjelaskan kondisi perluasan pada <<extend>>



Relasi <<extend>> berarti use case tambahan hanya terjadi dalam kondisi tertentu. Maka syarat atau kondisi yang menyebabkan perluasan itu

terjadi harus dijelaskan. Jika tidak disebutkan maka diagram jadi ambigu dan pembaca tidak tahu kapan use case tambahan dijalankan.

c. Mengabaikan relasi antar usecase yang saling bergantung

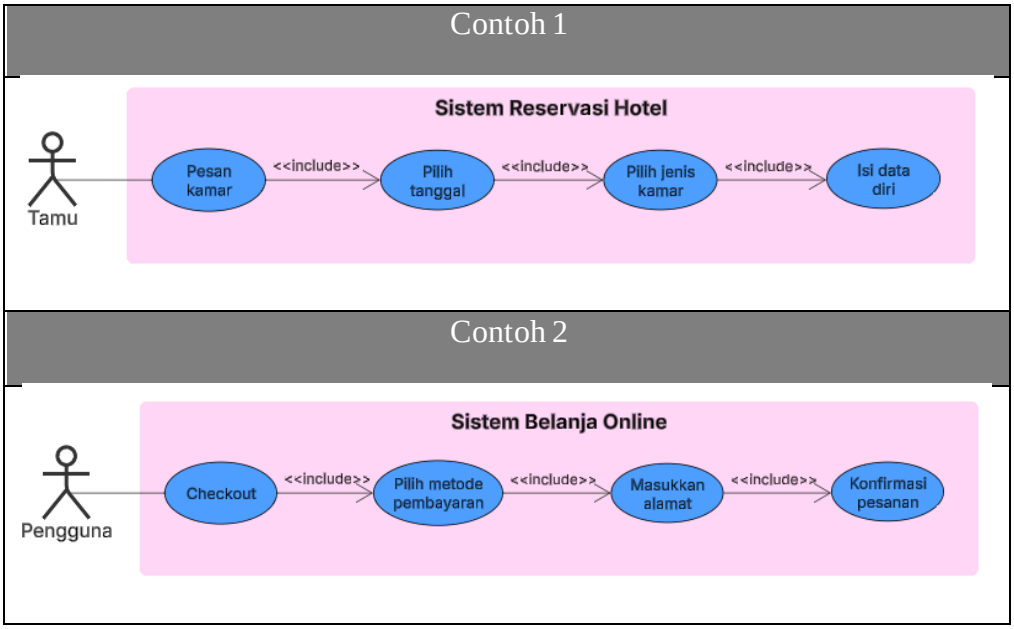


Ketika dua atau lebih use case berkaitan erat (bergantung satu sama lain), kita harus menggambarkan hubungan itu menggunakan:

- <<include>> → jika satu use case selalu membutuhkan yang lain
- <<extend>> → jika opsional tergantung kondisi tertentu

Jika relasi tidak digambarkan, maka diagram jadi terkesan acak dan tidak modular. Selain itu, diagram menjadi sulit dipahami bagaimana proses sistem bekerja

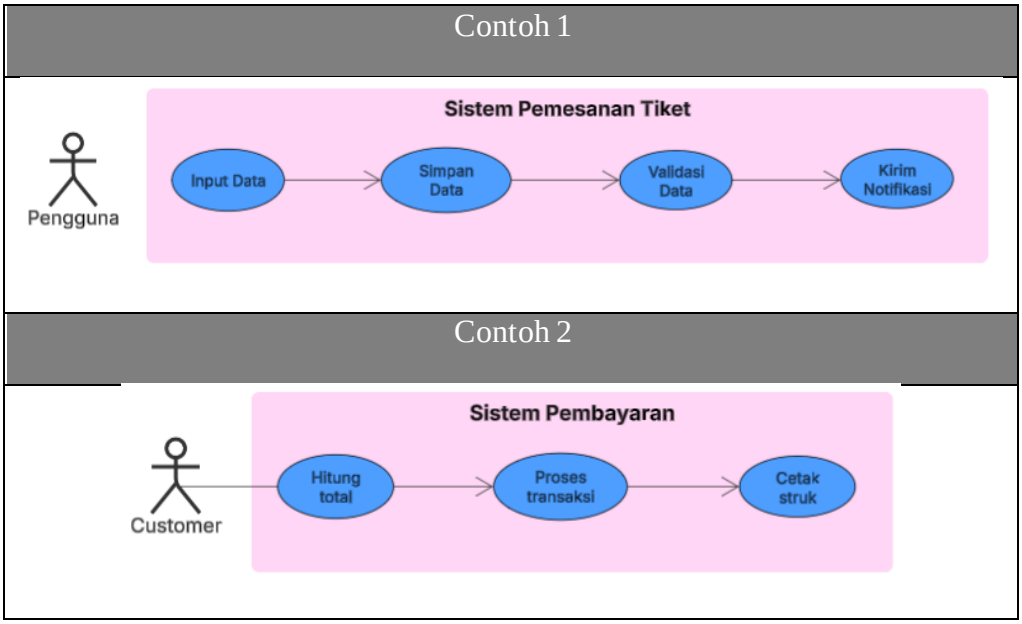
d. Terlalu banyak menggunakan <<include>> tanpa alasan



Relasi <<include>> digunakan hanya jika suatu use case selalu terjadi dalam skenario utama dan bagian yang disertakan tersebut digunakan ulang (reusable) di lebih dari satu use case. Jika memakai <<include>> secara berlebihan, maka diagram jadi terlalu rumit tanpa manfaat, relasi antar use case menjadi tidak bermakna, dan pemahaman alur sistem membingungkan. Dapat dilihat pada contoh, seharusnya bisa hanya menggunakan use case "pesan kamar" dan "Checkout" dikarenakan use case lain tidak reusable dan bisa dimasukkan ke use case tersebut.

5.5. Kesalahan Fokus dan Ruang Lingkup Diagram

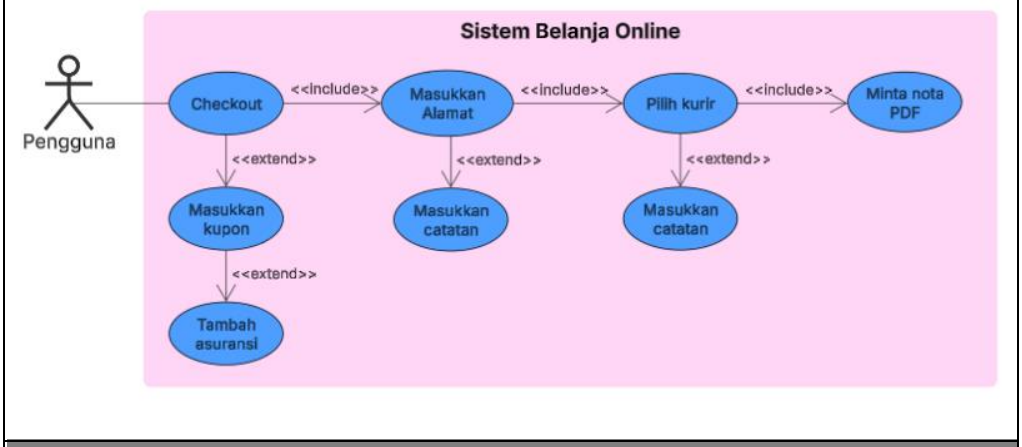
a. Menggambarkan alur proses, bukan interaksi aktor dengan sistem



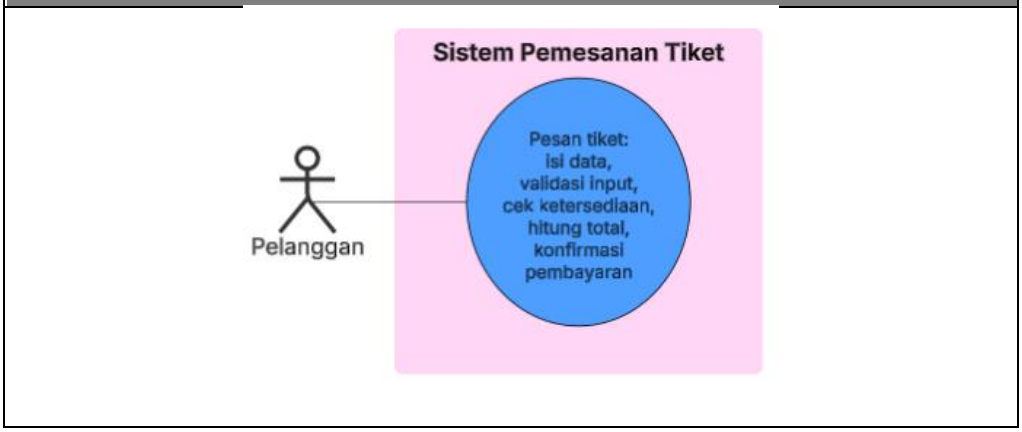
Use case diagram bukan flowchart. Tujuannya bukan untuk menggambarkan urutan proses atau logika langkah demi langkah dalam sistem, melainkan untuk menggambarkan interaksi antara aktor dan sistem, yaitu apa yang aktor lakukan terhadap sistem, dan apa yang sistem sediakan untuk aktor. Ketika use case diagram digunakan untuk menggambarkan alur kerja (workflow) seperti urutan aktivitas atau tahapan teknis sistem, maka fungsinya menjadi kabur, dan diagram tidak lagi mencerminkan tujuan utamanya sebagai alat komunikasi fungsionalitas sistem dari perspektif pengguna.

b. Terlalu banyak detail pada usecase diagram

Contoh 1

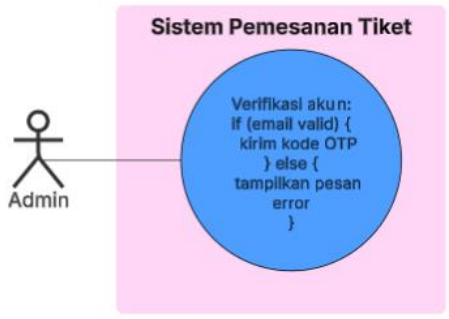


Contoh 2



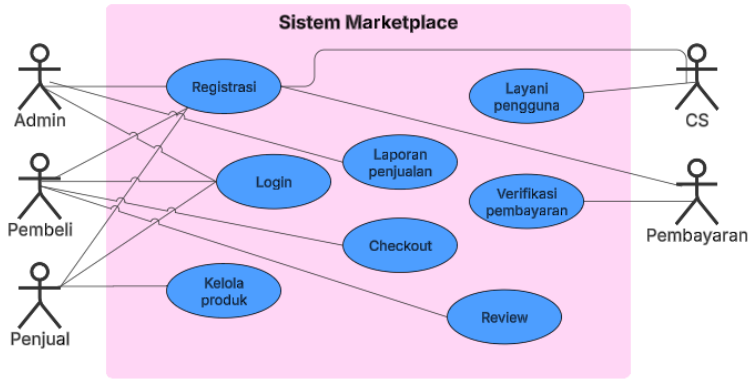
Use case diagram seharusnya sederhana dan fokus pada hubungan antara aktor dan sistem. Jika terlalu banyak informasi teknis, langkah-langkah internal, atau catatan proses dimasukkan ke dalam diagram, maka use case diagram menjadi rumit**,** sulit dibaca, dan tidak sesuai fungsinya sebagai alat komunikasi fungsionalitas tinggi.

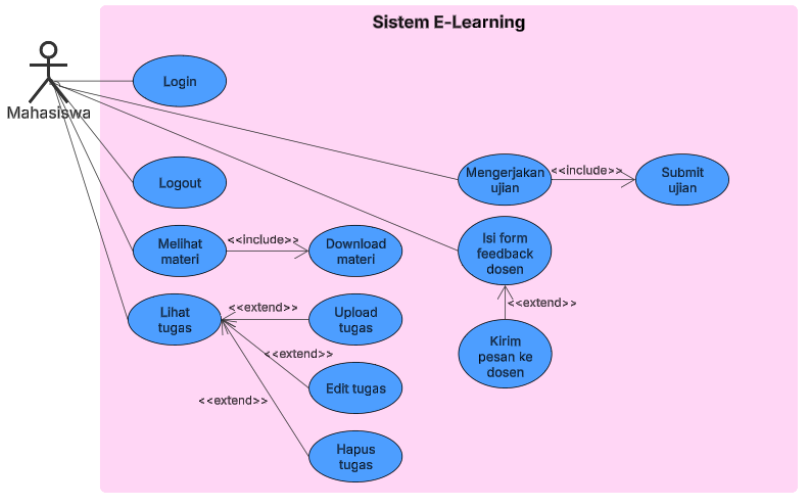
c. Menambahkan logika program ke dalam usecase

Contoh 1	Contoh 2
	

Use case diagram berfungsi untuk menjelaskan apa yang dilakukan sistem dari sudut pandang pengguna, bukan bagaimana sistem melakukannya. Ketika logika program seperti percabangan, perulangan, algoritma, atau kode dimasukkan ke dalam use case, diagram menjadi tidak sesuai tujuannya dan malah membingungkan pembacanya.

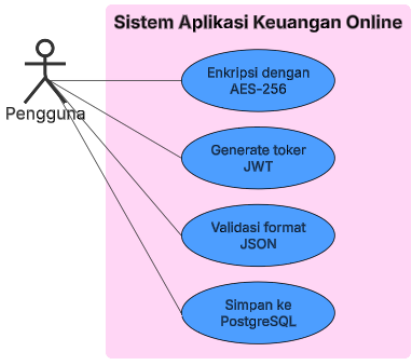
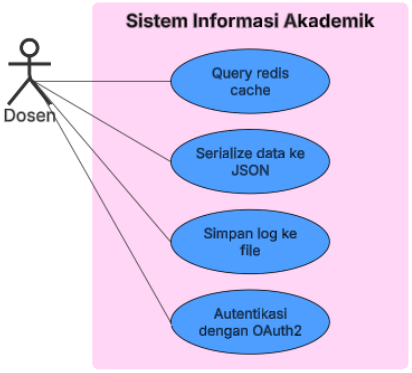
d. Membuat diagram terlalu kompleks dan besar

Contoh 1


Contoh 2


Use case diagram yang terlalu kompleks, misalnya dengan terlalu banyak aktor, terlalu banyak use case, dan terlalu banyak relasi, akan sulit dibaca, dipahami, dan dipelihara. Tujuan utama use case diagram adalah memberikan gambaran umum interaksi pengguna dengan sistem, bukan menjelaskan seluruh detail sistem dalam satu halaman.

e. Terlalu fokus pada aspek teknis sistem

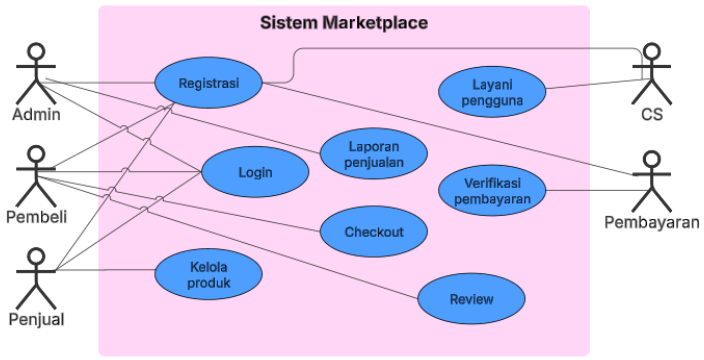
Contoh 1	Contoh 2
	

Use case diagram seharusnya menjelaskan interaksi antara aktor dan sistem, bukan detail teknis implementasi seperti algoritma, protokol, database, atau arsitektur sistem. Ketika terlalu fokus pada aspek teknis, diagram menjadi sulit dipahami oleh stakeholder non-teknis dan kehilangan fungsinya sebagai alat komunikasi kebutuhan fungsional.

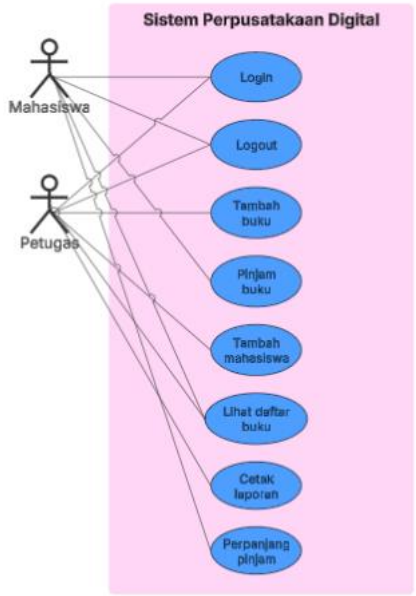
5.6. Kesalahan dalam Pemeliharaan dan Organisasi (Pengelompokan) Sistem

a. Tidak mengelompokkan usecase berdasarkan fungsi sistem

Contoh 1

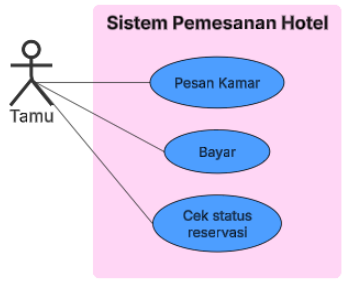
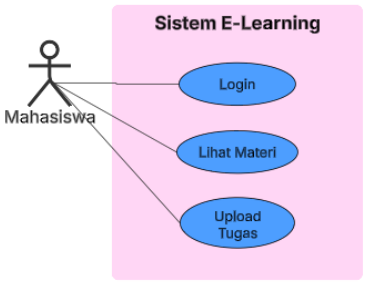


Contoh 2



Use case diagram yang tidak dikelompokkan berdasarkan fungsionalitas sistem akan terlihat berantakan, sulit dipahami, dan membingungkan stakeholder. Setiap sistem biasanya memiliki beberapa fitur utama, dan use case sebaiknya dikelompokkan sesuai fitur atau modul seperti: manajemen pengguna, akademik, transaksi, laporan, dll.

b. Tidak memperbarui diagram seiring perubahan kebutuhan

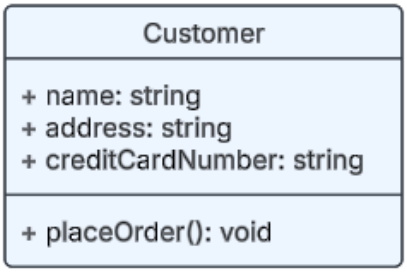
Contoh 1	Contoh 2
Awalnya sistem hanya dibutuhkan untuk pemesanan kamar  Kemudian ada penambahan fitur layanan spa dan restoran tetapi use case diagram tidak diperbarui	Sistem awal hanya untuk akses materi dan tugas  Kemudian universitas menambahkan fitur "forum diskusi" dan "webinar langsung" tetapi use case diagram tidak diperbarui

Use case diagram adalah dokumen hidup yang harus diperbarui setiap kali ada perubahan kebutuhan sistem**.** Jika tidak diperbarui, diagram menjadi tidak relevan, membingungkan tim pengembang, dan bisa menyebabkan miskomunikasi antar stakeholder.

6. KESALAHAN UMUM DALAM PEMBUATAN CLASS DIAGRAM

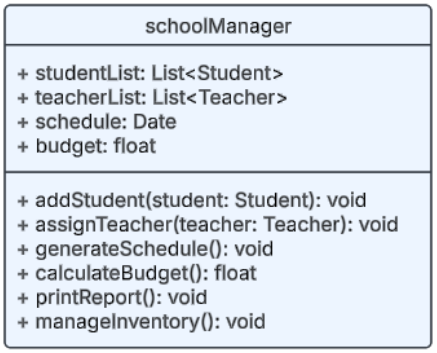
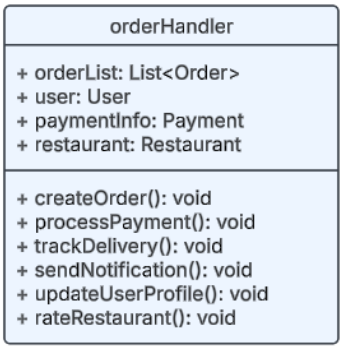
6.1. Kesalahan dalam Prinsip Desain Objek

a. Mengabaikan enkapsulasi

Contoh 1	Contoh 2
	

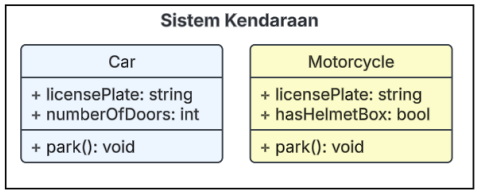
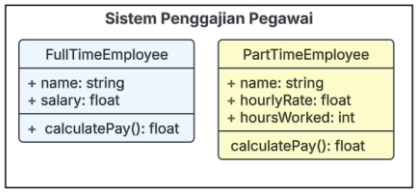
Atribut bersifar public (+) padahal ada atribut sensitive seperti stock, address, dan creditCardNumber.

b. Terlalu banyak tanggung jawab

Contoh 1	Contoh 2
	

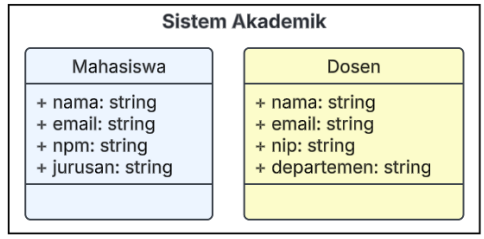
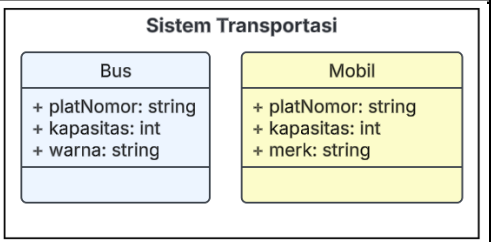
Semua tanggung jawab yang seharusnya dimiliki oleh class berbeda digabung menjadi satu class, yang membuatnya sulit dirawat dan diperluas. Class ini melanggar prinsip SRP (Single Responsibility Principle) di OOP karena tidak punya fokus tanggung jawab tunggal. Jika ada perubahan pada manajemen inventaris, class ini ikut terdampak, padahal tidak relevan dengan manajemen siswa/guru.

c. Tidak menggunakan generalisasi

Contoh 1	Contoh 2
	

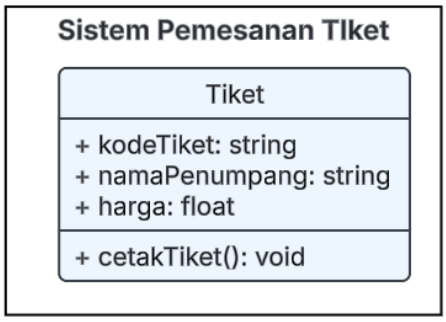
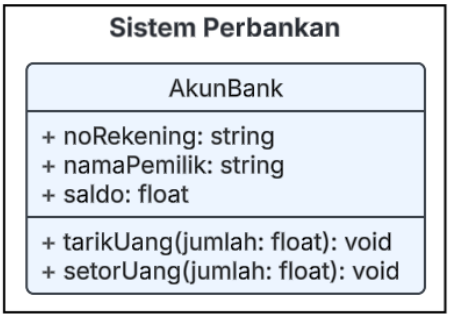
Kedua contoh di atas tidak memanfaatkan generalisasi (inheritance) meskipun terdapat atribut dan method yang sama antar class, seperti licensePlate dan park() di kendaraan, atau name dan calculatePay() di pegawai.

d. Duplikasi atribut

Contoh 1	Contoh 2
	

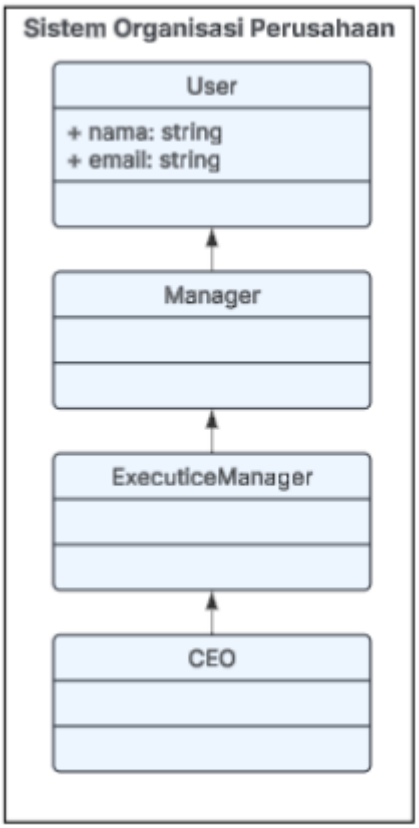
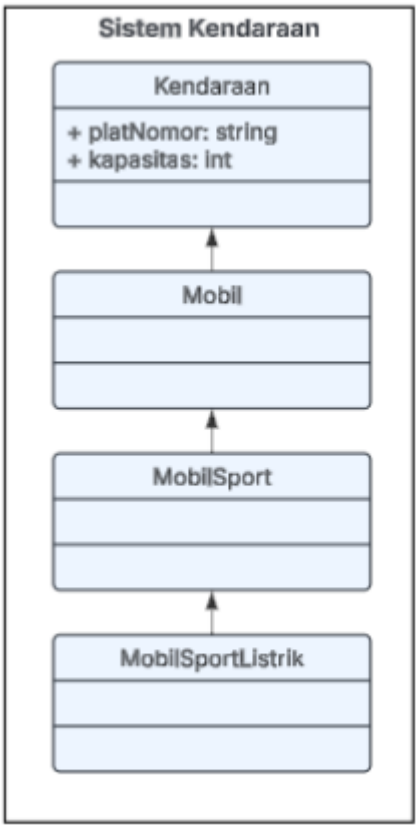
Kesalahan pada kedua contoh di atas terletak pada adanya atribut-atribut yang duplikatif di beberapa class, seperti nama dan email yang muncul di class Mahasiswa dan Dosen, serta platNomor dan kapasitas yang muncul di class Bus dan Mobil. Duplikasi atribut ini menunjukkan bahwa desain tidak efisien dan mengabaikan prinsip reuse (penggunaan kembali) dalam pemrograman berorientasi objek. Masalah ini dapat dihindari dengan memperkenalkan superclass atau abstract class yang menampung atribut-atribut umum, seperti class Person untuk Mahasiswa dan Dosen, atau class Kendaraan untuk Bus dan Mobil. Dengan begitu, struktur class menjadi lebih bersih, memudahkan pemeliharaan, dan mendukung pewarisan atribut serta metode yang lebih tepat guna.

e. Menjadikan semua atribut public

Contoh 1	Contoh 2
	

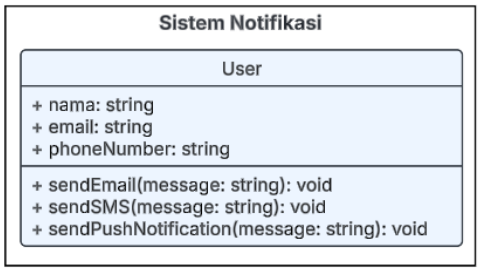
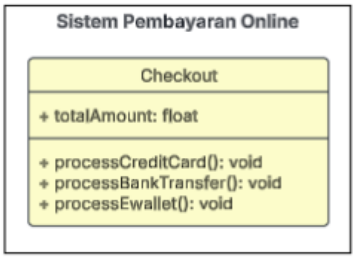
Menjadikan semua atribut publik melanggar prinsip *enkapsulasi*, karena data internal bisa diakses dan diubah bebas dari luar class. Ini berisiko pada keamanan dan konsistensi data. Atribut sebaiknya dibuat private, lalu diakses melalui method getter dan setter untuk menjaga kontrol.

f. Terlalu banyak pewarisan

Contoh 1	Contoh 2
	

Terlalu banyak level pewarisan menyebabkan hierarki menjadi dalam dan sulit dipelihara. Ini melanggar prinsip desain yang menganjurkan pewarisan yang ringkas dan jelas. Jika pewarisan terlalu dalam, perubahan pada superclass bisa berdampak luas dan sulit dilacak. Solusinya adalah menggunakan komposisi atau interface, bukan selalu inheritance, untuk membagi tanggung jawab.

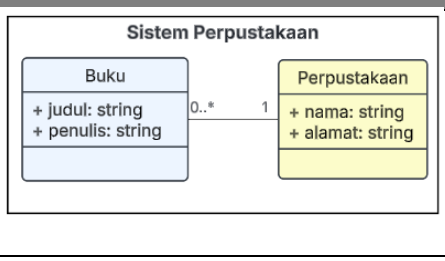
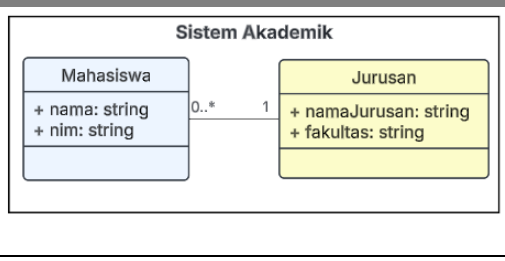
g. Tidak memperhatikan pola desain

Contoh 1	Contoh 2
	

Pada kedua contoh di atas, class memiliki banyak method spesifik untuk berbagai jenis aksi, seperti sendEmail, sendSMS, atau processCreditCard, processEwallet, dan sebagainya. Ini menunjukkan bahwa desain tidak menggunakan pola desain yang sesuai, seperti Strategy Pattem atau Factory Pattern. Akibatnya, setiap penambahan jenis notifikasi atau metode pembayaran baru akan membuat class semakin besar dan sulit dipelihara. Seharusnya, masing-masing jenis aksi dibuat sebagai class tersendiri yang mengimplementasikan interface atau strategi umum, sehingga sistem lebih fleksibel, modular, dan mudah diperluas.

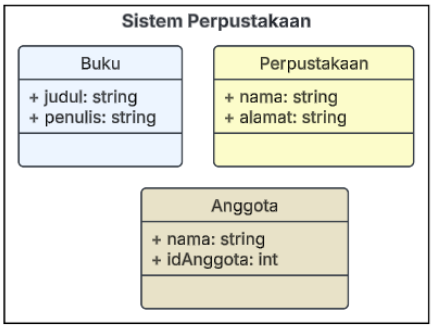
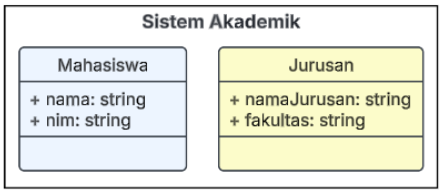
6.2. Kesalahan dalam Penggunaan Relasi Antar Kelas

a. Penggunaan asosiasi yang tidak perlu

Contoh 1	Contoh 2
	

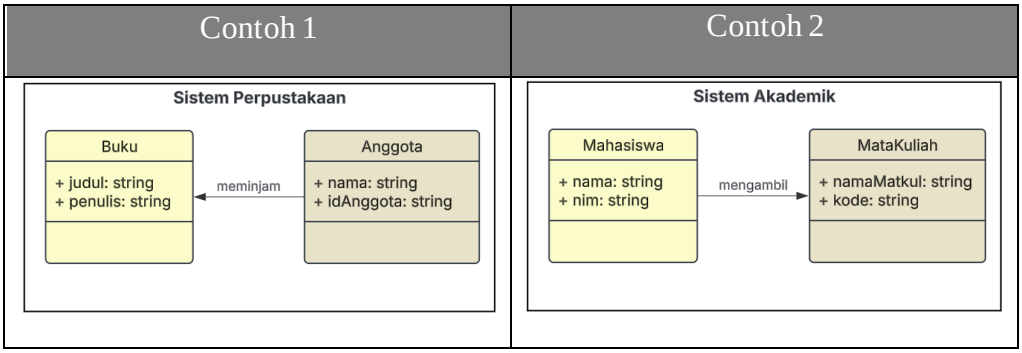
Kesalahan dalam kedua contoh di atas adalah penggunaan asosiasi antar class yang tidak mencerminkan hubungan yang logis atau dibutuhkan dalam konteks sistem. Misalnya, dalam contoh pertama, Buku tidak perlu memiliki relasi langsung ke Perpustakaan jika hubungan itu sudah dicakup dalam manajemen koleksi atau database sistem. Begitu pula, Jurusan tidak harus memiliki asosiasi langsung ke Mahasiswa jika data mahasiswa cukup dikelola melalui ID atau relasi tidak langsung. Penggunaan asosiasi yang tidak perlu membuat diagram lebih kompleks dan membingungkan, serta menyulitkan pengembangan dan pemeliharaan sistem karena relasi antar class tidak relevan secara fungsional.

b. Tidak menyediakan relasi yang jelas

Contoh 1	Contoh 2
	

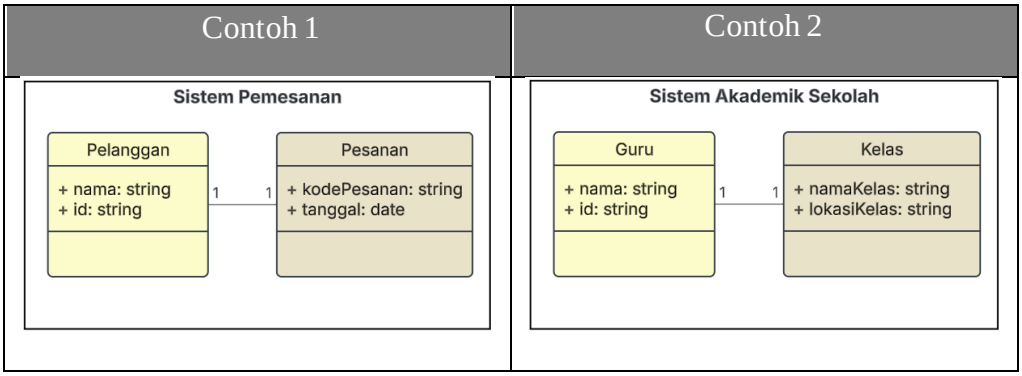
Kesalahan ini terjadi ketika class-class dibuat tanpa relasi apa pun, padahal fungsionalitas sistem sangat bergantung pada interaksi antar objek. Class diagram yang tidak menyertakan relasi menjadikan desain sistem terkesan terpisah-pisah (terisolasi), sulit dipahami, dan tidak mewakili proses bisnis sebenarnya. Relasi yang jelas seperti asosiasi sangat penting untuk menunjukkan bagaimana data dan objek saling terhubung dalam sistem.

c. Mengbaikan multiplicity



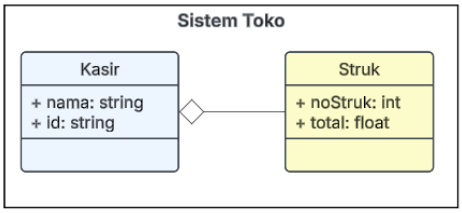
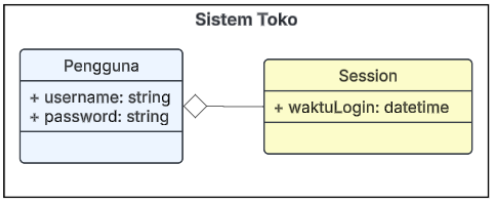
Multiplicity penting dalam class diagram untuk menjelaskan berapa banyak objek dari satu class yang terhubung dengan objek dari class lain. Tanpa multiplicity, relasi menjadi ambigu dan bisa disalahartikan. Hal ini dapat menyebabkan kesalahan saat implementasi, seperti mengira hubungan bersifat satu-satu padahal sebenarnya satu-ke-banyak. Menyediakan multiplicity membuat desain sistem lebih jelas, terarah, dan sesuai dengan logika bisnis.

d. Hubungan yang tidak simetris



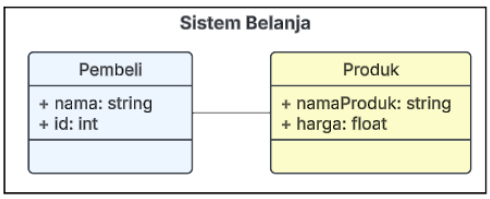
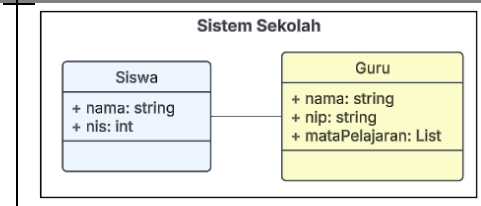
Kesalahan ini terjadi saat arah hubungan antar class tidak mencerminkan logika dunia nyata. Satu pelanggan bisa memiliki banyak pesanan, tapi diagram membatasi hanya satu. Hubungan menunjukkan satu kelas bisa diajar oleh banyak guru, tapi setiap guru hanya mengajar satu kelas. Ini tidak simetris dengan kenyataan, biasanya satu guru mengajar banyak kelas. Hubungan tidak simetris membuat interpretasi sistem menjadi keliru.

e. Menggunakan agregasi di tempat yang tidak perlu

Contoh 1	Contoh 2
	

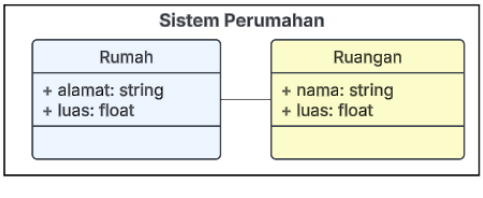
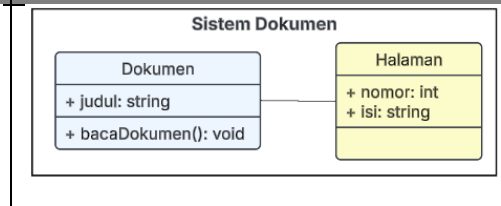
Agregasi menunjukkan hubungan "memiliki" yang lemah di mana objek bagian bisa tetap hidup tanpa induknya. Kesalahan terjadi ketika agregasi digunakan padahal objek tidak punya hubungan kepemilikan logis atau jangka panjang. Dalam kasus seperti Struk atau Session, asosiasi biasa sudah mencukupi dan lebih tepat secara desain.

f. Tidak menamai relasi

Contoh 1	Contoh 2
	

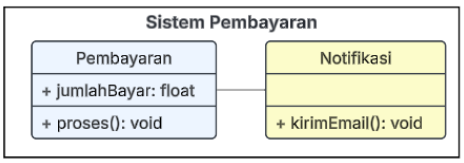
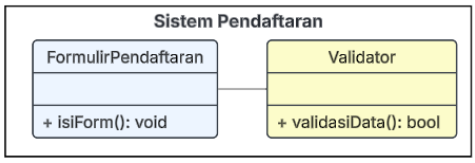
Tidak menamai relasi membuat class diagram sulit dipahami karena arah dan makna hubungan antar class menjadi ambigu. Nama relasi penting untuk menunjukkan peran atau fungsi hubungan tersebut dalam konteks sistem. Memberi nama seperti memesan, mengajar, mengelola, atau milik memperjelas maksud hubungan antar objek dalam desain.

g. Tidak menggambarkan komposisi saat diperlukan

Contoh 1	Contoh 2
	

Kesalahan ini terjadi saat relasi antar class tidak menunjukkan ketergantungan penuh antar objek. Dalam kasus seperti Rumah–Ruangan atau Dokumen–Halaman, objek bagian tidak bisa hidup mandiri. Oleh karena itu, relasi seharusnya digambarkan sebagai komposisi (belah ketupat hitam) untuk menunjukkan bahwa objek induk mengontrol siklus hidup objek bagian sepenuhnya. Tanpa komposisi, hubungan penting ini jadi tidak tersampaikan dalam desain.

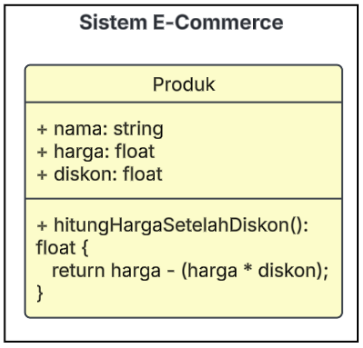
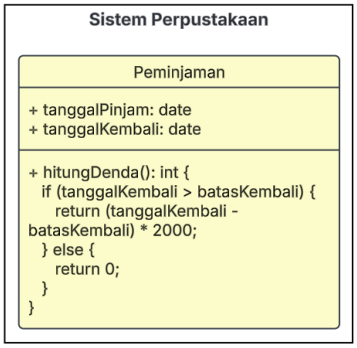
h. Tidak mengidentifikasi dependency

Contoh 1	Contoh 2
	

Kesalahan ini terjadi saat hubungan ketergantungan sementara antar class tidak ditunjukkan dalam diagram. Dependency penting untuk memperlihatkan bahwa satu class menggunakan/memanggil class lain, tanpa memiliki atau menyimpannya sebagai atribut. Dependency dapat disimbolkan dengan panah putus putus. Tanpa dependency, alur pemanggilan logika antar class menjadi tidak terlihat, dan dapat menyebabkan kesalahpahaman saat implementasi.

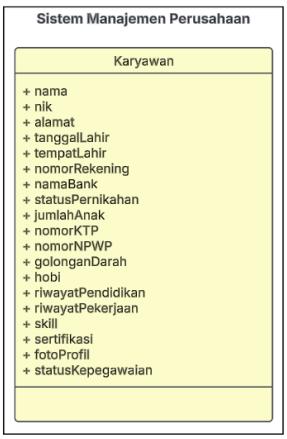
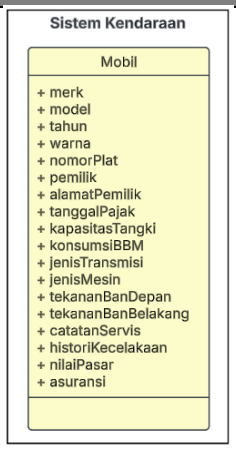
6.3. Kesalahan dalam Struktur Diagran dan Representasi Informasi

a. Menambahkan logika bisnis dalam class diagram

Contoh 1	Contoh 2
	

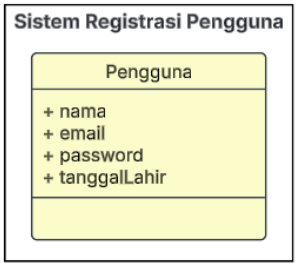
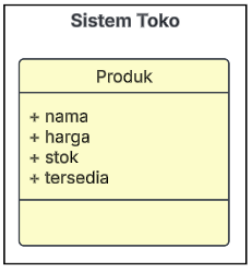
Class diagram berfungsi untuk mendeskripsikan struktur sistem berupa class, atribut, method, dan relasi — bukan tempat untuk menuliskan detail logika bisnis atau algoritma. Menambahkan logika bisnis (seperti perhitungan, pengkondisian, atau algoritma) membuat class diagram menjadi tidak fokus, terlalu rumit, dan keluar dari tujuannya sebagai alat dokumentasi desain sistem secara konseptual. Detail semacam itu sebaiknya disimpan untuk kode program atau diagram aktivitas.

b. Terlalu banyak atribut dalam satu class

Contoh 1	Contoh 2
	

Class dengan terlalu banyak atribut menandakan bahwa class tersebut memiliki tanggung jawab berlebih dan melanggar prinsip Single Responsibility Principle (SRP). Ini membuat sistem sulit dipelihara dan diperluas. Solusi yang tepat adalah memecah class menjadi beberapa class kecil yang lebih spesifik dan modular, serta mengelompokkannya secara logis.

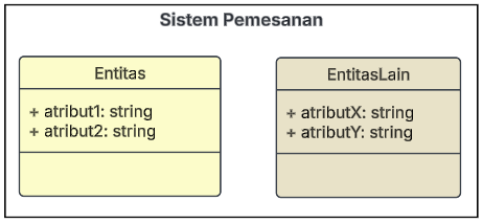
c. Tidak menentukan tipe data atribut

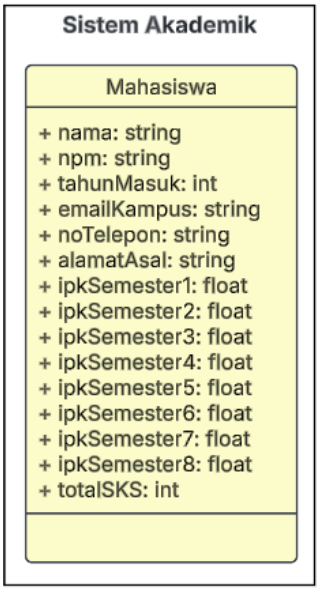
Contoh 1	Contoh 2
	

Tidak menyertakan tipe data atribut membuat class diagram tidak informatif dan sulit dipahami. Tipe data sangat penting untuk menjelaskan bagaimana data disimpan dan diproses. Tanpa ini, pengembang bisa salah menafsirkan maksud atribut, sehingga rawan kesalahan saat implementasi.

d. Terlalu abstrak atau terlalu detail

Contoh 1	Contoh 2
Terlalu Abstrak	Terlalu Detail

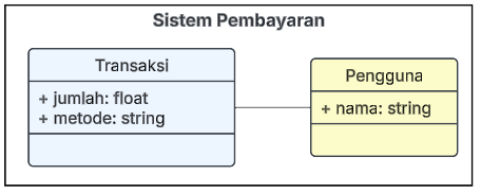


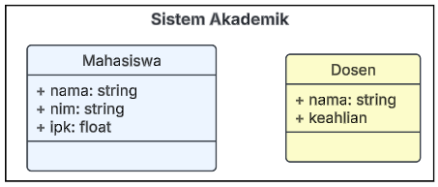


Kesalahan ini muncul saat class diagram tidak seimbang antara abstraksi dan detail. Jika terlalu abstrak, diagram menjadi tidak jelas dan sulit diterapkan. Jika terlalu detail, diagram jadi rumit, sulit dipelihara, dan melanggar prinsip pemisahan tanggung jawab. Class diagram yang baik harus fokus pada struktur inti sistem dengan level detail yang relevan dan proporsional.

e. Tidak menambahkan keterangan tambahan

Contoh 1	Contoh 2

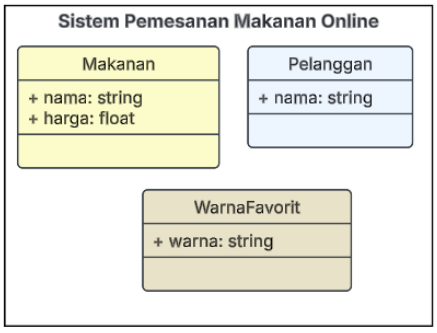
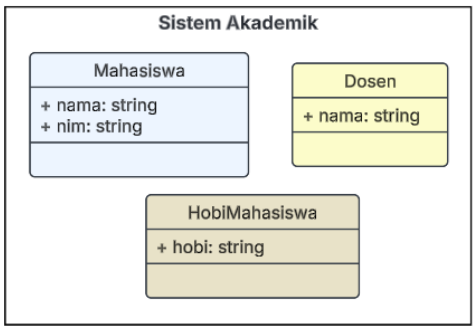




Class diagram tanpa keterangan tambahan dapat membingungkan pembaca, terutama jika sistem kompleks atau memiliki istilah khusus. Keterangan (dalam bentuk catatan, stereotype, constraints, atau asumsi) penting untuk menjelaskan maksud atribut, relasi, aturan, atau peran. Pada contoh, tidak diberi keterangan bagaimana perhitungan ipk. Selain itu, jumlah juga tidak diberi keterangan, apakah jumlah barang, nominal, atau unit lain. Tanpa ini, class diagram jadi kurang komunikatif dan mudah disalahartikan saat pengembangan sistem.

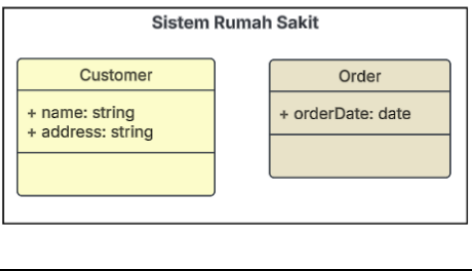
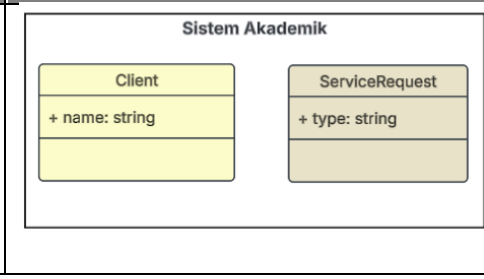
6.4. Kesalahan dalam Pemilihan dan Pengogranisasian Class

a. Membuat class yang tidak relevan

Contoh 1	Contoh 2
	

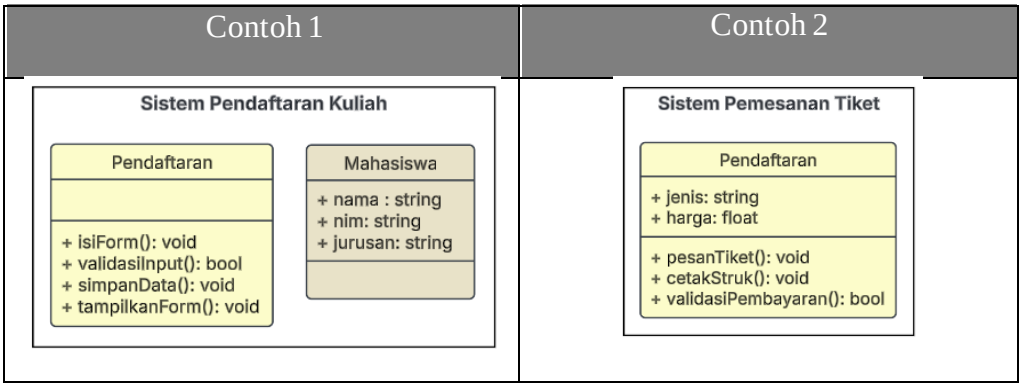
Kesalahan ini terjadi ketika class yang tidak berkaitan langsung dengan kebutuhan sistem tetap dimasukkan dalam diagram. Akibatnya, class diagram jadi tidak fokus dan sulit dipahami. Warna favorit tidak ada hubungannya dengan pemesanan makanan, dan hobi mahasiswa tidak ada hubungannya dengan aktivitas akademik.

b. Tidak konsisten dengan domain

Contoh 1	Contoh 2
	

Ketidakkonsistenan dengan domain menyebabkan kebingungan dan menunjukkan kurangnya pemahaman terhadap konteks sistem. Pemilihan nama class, atribut, dan relasi harus mencerminkan istilah nyata yang digunakan dalam dunia nyata sesuai domainnya. Hal ini penting agar diagram mudah dipahami oleh stakeholder, analis, dan developer yang bekerja dalam sistem tersebut.

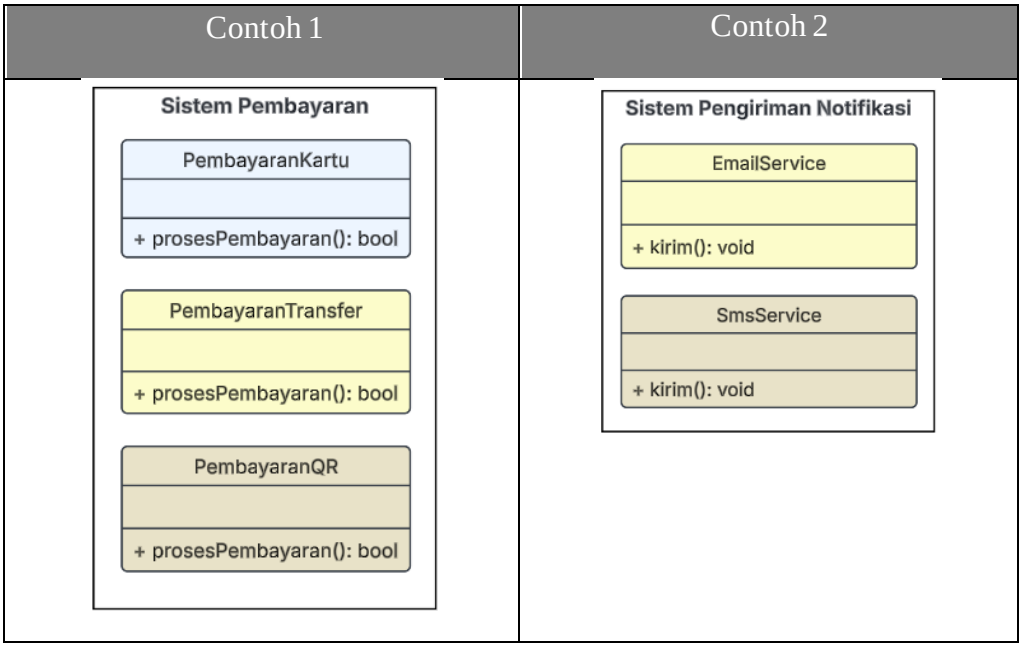
c. Tidak memisahkan class, entitas, boundary, dan control



Pada contoh 1, Class Pendaftaran mencampur logika proses (control), tampilan (boundary), dan penyimpanan data (entitas) dalam satu tempat. Ini melanggar prinsip separasi tanggung jawab, sehingga sulit dipelihara dan diperluas. Pada contoh 2, Semua aktivitas, mulai dari menyimpan data, memproses pemesanan, hingga menampilkan output, dilakukan dalam satu class Tiket. Ini mencampur entitas, control, dan boundary menjadi satu. Dalam desain yang baik, terutama dengan pendekatan Model-View-Control (MVC) atau Entity-Boundary-Control (EBC), setiap class harus memiliki tanggung jawab yang jelas.

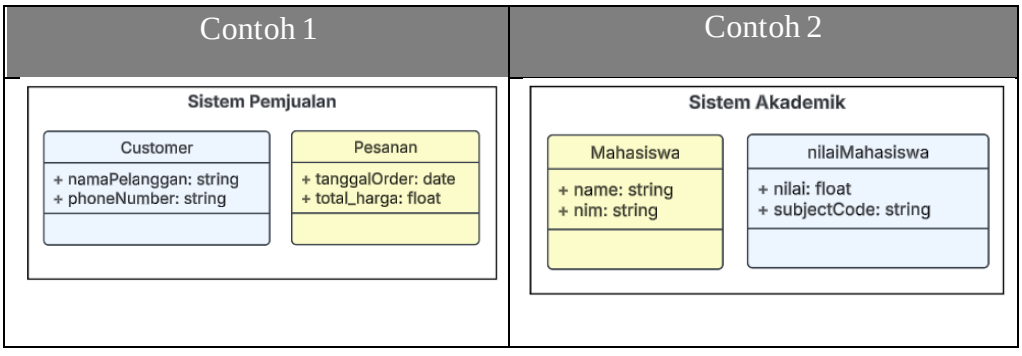
6.5. Kesalahan dalam Penamaan dan Konsistensi

a. Tidak menggunakan interface



Interface penting dalam class diagram untuk menstandarkan perilaku antar class yang berbeda tapi memiliki method serupa. Tanpa interface, desain menjadi kurang fleksibel dan tidak mendukung polimorfisme, yang penting untuk scalability dan maintainability. Interface juga membantu dokumentasi dengan memperjelas kontrak antar class, sehingga developer tahu fungsi apa yang harus diimplementasikan.

b. Penggunaan nama yang tidak konsisten



Penggunaan nama yang tidak konsisten dalam class diagram menyebabkan kebingungan, mengurangi keterbacaan, dan meningkatkan risiko error dalam implementasi kode. Nama class dan atribut sebaiknya mengikuti konvensi penamaan yang seragam, baik dari segi bahasa (semua Indonesia atau semua Inggris) maupun gaya penulisan (misalnya camelCase atau snake_case), agar sistem lebih mudah dipahami dan dikelola.

7. KESIMPULAN

Setelah melakukan tugas akhir untuk mata kuliah aplikasi mobile, terdapat beberapa kesimpulan yang di dapati

Kesalahan dalam membuat activity diagram dapat mengakibatkan kesalahpahaman dalam analisis dan implementasi sistem. Mulai dari tidak adanya titik awal/akhir, alur kontrol yang tidak logis, representasi aktivitas yang tidak jelas, pembagian peran yang rancu dalam swimlane, hingga penggunaan notasi yang tidak sesuai standar UML—semuanya dapat membuat diagram sulit dipahami dan tidak representatif terhadap proses sebenarnya. Oleh karena itu, penting untuk memperhatikan setiap aspek struktur, alur, notasi, dan peran agar activity diagram benar-benar berfungsi sebagai alat komunikasi yang efektif dalam pengembangan sistem.

Kesalahan dalam membuat sequence diagram umumnya terbagi ke dalam lima klasifikasi utama, mulai dari representasi objek, penggambaran pesan, penggunaan fragment, fokus dan ruang lingkup, hingga ketidaksesuaian dengan diagram lain atau standar UML. Kesalahan representasi objek seperti penggunaan nama yang tidak konsisten, tidak adanya lifeline, hingga self-message yang ambigu dapat mengacaukan pemahaman terhadap objek yang terlibat. Pada penggambaran pesan, kesalahan arah panah, urutan, atau penggunaan pesan tanpa makna memperburuk kejelasan komunikasi antar objek. Fragment seperti alt, loop, dan opt sering digunakan secara tidak tepat, padahal sangat penting untuk menampilkan logika kondisi atau pengulangan. Selain itu, sequence diagram menjadi tidak efektif bila mencakup terlalu banyak objek, logika, atau bahkan proses yang tidak relevan. Terakhir, diagram ini harus selaras dengan use case dan standar UML, termasuk penggunaan activation bar dan alur waktu yang logis. Oleh karena itu, pemahaman dan penerapan kaidah UML secara konsisten sangat penting agar sequence diagram benar-benar mampu menggambarkan skenario sistem secara akurat, ringkas, dan mudah dipahami.

Use case diagram sering kali dibuat dengan berbagai kesalahan yang mengurangi fungsinya sebagai alat komunikasi kebutuhan sistem. Kesalahan umum meliputi penggunaan aktor yang tidak tepat (seperti database atau aktor fiktif), struktur diagram yang tidak sesuai standar (seperti tidak menyertakan nama sistem atau boundary), penamaan use case yang tidak representatif (menggunakan kata benda atau menyatukan banyak fungsi dalam satu simbol), serta kesalahan dalam penggunaan

relasi <<include>> dan <<extend>>. Diagram juga kerap terlalu fokus pada aspek teknis, menampilkan logika program, atau menjadi terlalu kompleks karena memuat terlalu banyak detail. Selain itu, banyak diagram yang tidak diperbarui saat kebutuhan sistem berubah dan tidak mengelompokkan use case berdasarkan fungsi sistem, sehingga menyulitkan pembacaan dan pemeliharaan. Untuk itu, use case diagram harus disusun secara jelas, modular, dan tetap fokus pada interaksi antara aktor dan sistem.

Class diagram adalah salah satu jenis diagram UML yang digunakan untuk memodelkan struktur statis dari sistem dengan menggambarkan class, atribut, method, dan relasi antar class. Diagram ini membantu dalam memahami desain sistem dari sisi objek dan hubungan antar komponennya. Berdasarkan berbagai contoh yang telah dibahas, dapat disimpulkan bahwa kesalahan dalam membuat class diagram umumnya terjadi karena kurangnya pemahaman terhadap prinsip desain berorientasi objek, struktur relasi antar class, dan representasi informasi yang tepat. Kesalahan seperti mengabaikan enkapsulasi, membuat class terlalu besar atau terlalu abstrak, hingga tidak konsisten dalam penamaan atau penggunaan interface, dapat menyebabkan diagram sulit dipahami, tidak fleksibel, dan tidak mencerminkan kebutuhan sistem secara akurat. Untuk menghasilkan class diagram yang baik, penting untuk menjaga konsistensi, relevansi, keterpisahan peran (entitas, boundary, control), serta menyertakan detail yang cukup tanpa membebani diagram dengan logika atau atribut yang tidak perlu.